

MASTER REVIEWER

Data Mining & Recommender Systems

Frequent Itemset Mining • Association Rules

Neighborhood-based Collaborative Filtering

Latent-factor Collaborative Filtering

A complete, self-contained study guide
with worked examples, practice problems, and answer keys

Compiled from the course lecture slides (Sessions 3–10)
and the three Jupyter notebooks

Emphasis on manual pen-and-paper problem solving

Contents

How to use this reviewer	v
I Frequent Itemset Mining and Association Rule Mining	1
Part I — Topic Checklist	2
1 Foundations: Market-Basket Analysis and Frequent Itemsets	3
1.1 Motivation and the market-basket model	3
1.2 Itemsets and support	4
1.3 Support monotonicity and downward closure	5
2 Brute-Force Enumeration and Scalability	7
2.1 The brute-force algorithm	7
2.2 Complexity analysis	8
3 The Apriori Algorithm	9
3.1 Idea: breadth-first search with pruning	9
3.2 The three building blocks	9
3.3 Fully worked manual trace	11
3.4 The maximal-itemset variant <code>apriori2</code>	12
4 The FP-Growth Algorithm	13
4.1 Idea: compress the database into a tree	13
4.2 The FP-tree, header table, and node links	14
4.3 Fully worked manual trace of FP-Growth	14
4.4 Apriori vs. FP-Growth	16
5 Association Rule Mining	17
5.1 Definitions	17
5.2 Generating rules	18
5.3 Fully worked manual trace (the 13 rules)	18
6 The pyFIM Library and the Online Retail Case Study	20
6.1 Using pyFIM	20
6.2 Online Retail case study	21

7	Summary: Formulas, Algorithms, and Exam Strategy	22
7.1	Formula sheet (Part I)	22
7.2	Algorithm summaries	22
7.3	Decision guide	23
7.4	Common exam traps and time savers	23
8	Practice Problems — Part I	25
8.1	Category A — Conceptual	25
8.2	Category B — Basic computation	25
8.3	Category C — Manual algorithm execution	26
8.4	Category D — Multi-step / integrated	26
8.5	Category E — Challenging	26
II	Neighborhood-based Collaborative Filtering	27
Part II	Topic Checklist	28
9	Recommender Systems: The Utility Matrix and Approaches	29
9.1	The utility matrix	29
9.2	Explicit and implicit ratings	30
9.3	What makes a good recommendation	30
9.4	Content-based vs. collaborative filtering	31
9.5	Memory-based vs. model-based CF	31
10	Similarity Measures	32
10.1	The running toy matrix	32
10.2	Raw cosine similarity	33
10.3	Adjusted cosine = Pearson correlation	33
11	User-based Collaborative Filtering	35
11.1	The algorithm	35
11.2	The prediction formula	36
11.3	Fully worked manual example	36
11.4	The <code>recommend</code> function	37
11.5	The Jester dataset and notebook assertions	37
12	Item-based Collaborative Filtering	39
12.1	The algorithm	39
12.2	Fully worked manual example	40
12.3	User-based vs. item-based: the broken duality	40
13	Evaluating Recommender Systems	42
13.1	Online vs. offline evaluation	42
13.2	Desirable properties	42
13.3	Accuracy of ratings: MSE, RMSE, MAE, MAPE	43
13.4	Accuracy of ranking: DCG and NDCG	43
13.5	Notebook assertions for DCG/NDCG	44

14 Summary: Formulas, Algorithms, and Exam Strategy	46
14.1 Formula sheet (Part II)	46
14.2 Algorithm summaries	46
14.3 Decision guide	47
14.4 Common exam traps and time savers	47
15 Practice Problems — Part II	49
15.1 Category A — Conceptual	49
15.2 Category B — Basic computation	49
15.3 Category C — Manual algorithm execution	50
15.4 Category D — Multi-step / integrated	50
15.5 Category E — Challenging	50
III Latent-factor Collaborative Filtering	51
Part III — Topic Checklist	52
16 The Latent-Factor Model: UV Decomposition	53
16.1 From matrix completion to matrix factorization	53
16.2 The objective function	54
16.3 Gradients	54
17 Gradient Descent: Batch and Stochastic	55
17.1 Batch gradient descent	55
17.2 Stochastic gradient descent (SGD)	56
17.3 Small SGD trace	56
18 Alternating Least Squares (ALS)	57
18.1 The algorithm	57
18.2 Fully worked first iteration	58
18.3 The <code>als</code> function and recommendation	59
19 Coordinate Gradient Descent (CGD)	61
19.1 The idea	61
19.2 Worked example: one element of U , then one of V	61
19.3 The closed forms	62
19.4 Results and the storage advantage	63
20 The Surprise Library	64
20.1 The code, line by line	64
21 Summary: Formulas, Algorithms, and Exam Strategy	66
21.1 Formula sheet (Part III)	66
21.2 Algorithm summaries and decision guide	66
21.3 Common exam traps and time savers	67

22 Practice Problems — Part III	68
22.1 Category A — Conceptual	68
22.2 Category B — Basic computation	68
22.3 Category C — Manual algorithm execution	69
22.4 Category D — Multi-step / integrated	69
22.5 Category E — Challenging	69
 IV Answer Key to Practice Problems	 70
23 Part I — Frequent Itemset Mining: Solutions	71
24 Part II — Neighborhood-based Collaborative Filtering: Solutions	79
25 Part III — Latent-factor Collaborative Filtering: Solutions	85
Formula Sheet (Master)	90
Bibliography	92

How to use this reviewer

This reviewer is a *standalone textbook* covering every concept in the three course notebooks and the accompanying lecture slides (Sessions 3–10). It is organized into three parts:

1. **Part I — Frequent Itemset Mining and Association Rule Mining** (Sessions 3–4).
2. **Part II — Neighborhood-based Collaborative Filtering** (Sessions 5–7).
3. **Part III — Latent-factor Collaborative Filtering** (Sessions 8–10).

Each chapter follows the same skeleton: *learning objectives* → *intuition* → *formal definitions* → *derivations* → *step-by-step manual algorithms* → *fully worked examples with intermediate arithmetic* → *interpretation* → *common mistakes* → *connections to the notebook code*. Boxed callouts highlight the formulas you must memorize, the intuitions that make them click, the traps that cost marks, and the exam tricks that save time.

How the three pillars map to study. The reviewer distinguishes explicitly between what must be (i) *memorized* (e.g. the DCG formula, the support monotonicity statement), (ii) *conceptually understood* (e.g. why the Apriori pruning rule is safe, why we mean-center before computing similarity), and (iii) *procedurally fluent* (e.g. executing Apriori or FP-Growth by hand on a small database). Exam questions rarely test these in isolation, so each chapter’s worked examples and practice problems blend all three.

Practice problems. Every part ends with a bank of practice problems in five categories:

- **A — Conceptual:** definitions, properties, “why” questions.
- **B — Basic computation:** one-shot formula evaluation.
- **C — Manual algorithm execution:** trace an algorithm by hand.
- **D — Multi-step / integrated:** chain several procedures.
- **E — Challenging:** synthesis, proofs, edge cases, design.

Each problem is tagged with a difficulty (**Easy** / **Moderate** / **Difficult**) and an expected time budget. Full solutions — with reasoning, intermediate arithmetic, execution traces, verification checks, common wrong approaches, and point-based grading rubrics — appear in the *separate* answer key at the end of the book (Part IV), *not* immediately after each exercise. Attempt the problems before reading the solutions.

Notation and source fidelity. Where the slides and notebooks are ambiguous or appear inconsistent, the discrepancy is flagged explicitly in a *supplementary note*; the reviewer then adopts the convention implemented in the notebook (the authoritative, testable definition). Boxed statements marked “supplementary” are clarifications the author added because the source was silent; they are clearly labeled and never substitute for the source definitions.

Part I

Frequent Itemset Mining and Association Rule Mining

Part I — Topic Checklist

Tick each item once you can (a) state the definition from memory, (b) explain the intuition in your own words, and (c) carry out a small manual example.

- ☐ Market-basket model: database $\mathcal{T}$, transaction T_i , universe $\mathcal{U}$, dimensionality $d = |\mathcal{U}|$, binary matrix form.
- ☐ Itemset, k -itemset, subset lattice, and the number of k -itemsets $\binom{d}{k}$.
- ☐ Absolute support $\text{sup}(X)$, relative support $\text{relSup}(X)$, minimum support minsup , frequent itemset.
- ☐ Support monotonicity (downward closure) property and its proof.
- ☐ Maximal frequent itemsets and the notion of the frequent/infrequent border.
- ☐ Brute-force enumeration and its $O(N \cdot 2^d)$ complexity.
- ☐ Apriori: candidate generation, pruning via `remove_candidate`, level-by-level counting.
- ☐ FP-Growth: FP-tree, header table, node links, conditional pattern bases, conditional trees, ascending-support recursion.
- ☐ Association rules: antecedent, consequent, confidence, lift, and their interpretations.
- ☐ The pyFIM library: `supp` (positive vs. negative), `zmin`, `target`, `report`, `conf`.
- ☐ The Online Retail case study: cleaning, basket construction, statistics, and business recommendations.

Chapter 1

Foundations: Market-Basket Analysis and Frequent Itemsets

Learning objectives

After this chapter you should be able to:

- state the market-basket (transaction-database) model and its matrix form;
- define an itemset, a k -itemset, absolute support, relative support, and a frequent itemset;
- state and *prove* the support-monotonicity (downward-closure) property;
- define a maximal frequent itemset and the frequent/infrequent border;
- compute supports by hand on a small database.

1.1 Motivation and the market-basket model

Frequent Itemset Mining (FIM), also called *frequent pattern mining*, originated in *market-basket analysis*: a supermarket records, for each checkout, the set of products a single customer bought together. The store then asks questions such as: *which products are frequently bought together?*, *if a customer buys bread and milk, what else are they likely to buy?*, *which pairs of products should be placed on the same shelf or bundled in a promotion?*

Definition 1.1 (Transaction database). A *database* $\mathcal{T}$ is an unordered collection of N transactions $\{T_1, T_2, \dots, T_N\}$. Each transaction T_i is a set of $n_i = |T_i|$ items drawn from a *universe* $\mathcal{U}$ of all possible items. Let $d = |\mathcal{U}|$ be the number of distinct items.

The running example throughout Part I is a tiny grocery database with $N = 5$ transactions and universe $\mathcal{U} = \{\text{bread, butter, cheese, eggs, milk, yogurt}\}$ ($d = 6$):

tid	items
T_1	{bread, butter, milk}
T_2	{eggs, milk, yogurt}
T_3	{bread, cheese, eggs, milk}
T_4	{eggs, milk, yogurt}
T_5	{cheese, milk, yogurt}

The matrix form

Because a transaction is just a set, we can encode $\mathcal{T}$ as a $N \times d$ binary matrix: row i is transaction T_i , column j is item $u_j \in \mathcal{U}$, and the entry is 1 if $u_j \in T_i$ and 0 otherwise. The notebook's `to_sparse` function does exactly this and returns a sparse matrix (SciPy `csr_matrix`), because in real data d is huge and almost every entry is 0 — storing the zeros explicitly would waste memory.

Example 1.1 (Sparse-matrix encoding of the grocery database). Sort the universe alphabetically: $\mathcal{U} = [\text{bread, butter, cheese, eggs, milk, yogurt}]$. Then row 1 (transaction $T_1 = \{\text{bread, butter, milk}\}$) is

$$[1, 1, 0, 0, 1, 0],$$

because bread, butter, and milk are present while cheese, eggs, and yogurt are absent. The full matrix is a 5×6 matrix whose rows are $T_1, \dots, T_5$:

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

The notebook asserts exactly this shape — (5, 6) — and that row 0 equals $[1, 1, 0, 0, 1, 0]$.

1.2 Itemsets and support

Definition 1.2 (Itemset). An *itemset* X is a subset of the universe, $X \subseteq \mathcal{U}$. An itemset of cardinality $k = |X|$ is called a *k-itemset*.

For example the itemset $\{\text{bread, butter, milk}\}$ has 7 subsets: $\{\text{bread}\}$, $\{\text{butter}\}$, $\{\text{milk}\}$ (three 1-itemsets), $\{\text{bread, butter}\}$, $\{\text{bread, milk}\}$, $\{\text{butter, milk}\}$ (three 2-itemsets), and $\{\text{bread, butter, milk}\}$ itself (one 3-itemset).

Definition 1.3 (Absolute support). The (*absolute*) *support* $\text{sup}(X)$ of an itemset X is the number of transactions that contain X as a subset,

$$\text{sup}(X) = |\{T_i \in \mathcal{T} : X \subseteq T_i\}|.$$

Definition 1.4 (Relative support). The *relative support* $\text{relSup}(X)$ is the fraction of transactions containing X ,

$$\text{relSup}(X) = \frac{\text{sup}(X)}{N}.$$

Definition 1.5 (Frequent itemset). An itemset X is *frequent* (a *frequent pattern*) if its support is at least a user-specified *minimum support* $minsup$, i.e. $\text{sup}(X) \geq minsup$. The task of *frequent itemset mining* is to find *all* frequent itemsets in $\mathcal{T}$.

Absolute vs. relative support

Be careful: many references and libraries (including `pyFIM`, and parts of Aggarwal’s textbook) define “support” as the *relative* support, i.e. a fraction or percentage, whereas this notebook defines $\text{sup}(X)$ as the *absolute* count. Converting is easy — $\text{sup}(X) = N \cdot \text{relSup}(X)$ — but a mismatched convention is a very common source of wrong answers in exams.

Example 1.2 (Computing supports by hand). For the grocery database:

- $\text{sup}(\{\text{milk}\}) = 5$, since milk appears in *every* transaction; $\text{relSup}(\{\text{milk}\}) = 5/5 = 1$.
- $\text{sup}(\{\text{eggs}\}) = 3$ (in T_2, T_3, T_4).
- $\text{sup}(\{\text{bread, milk}\}) = 2$ (in T_1, T_3); $\text{relSup} = 2/5 = 0.4$.
- $\text{sup}(\{\text{bread, cheese}\}) = 1$ (only T_3).

1.3 Support monotonicity and downward closure

Theorem 1.1 (Support monotonicity). *For any itemsets $Y \subseteq X$, $\text{sup}(Y) \geq \text{sup}(X)$. Equivalently, the support of a subset is at least the support of the superset.*

Proof. Every transaction T_i that contains X also contains Y (because $Y \subseteq X$). Therefore the set of transactions containing X is a *subset* of the set of transactions containing Y ; a subset can never be larger than its superset, so $|\{T_i : X \subseteq T_i\}| \leq |\{T_i : Y \subseteq T_i\}|$, which is exactly $\text{sup}(X) \leq \text{sup}(Y)$. $\square$

Corollary 1.2 (Downward closure). *Every subset of a frequent itemset is itself frequent. Contrapositively, if X is infrequent, then every superset of X is also infrequent.*

Proof. If X is frequent then $\text{sup}(X) \geq minsup$; for any $Y \subseteq X$, $\text{sup}(Y) \geq \text{sup}(X) \geq minsup$, so Y is frequent. The contrapositive is the pruning rule actually used by Apriori (Section 3). $\square$

Why monotonicity matters

Support is *anti-monotone* with respect to set inclusion: adding more items to an itemset can only *shrink* the set of transactions that contain it (you need a customer who bought *all* of them). This single fact lets us prune the enormous search space: if an itemset is infrequent, we never need to check any of its supersets, because they are automatically infrequent.

Definition 1.6 (Maximal frequent itemset). A frequent itemset is *maximal* (at threshold $minsup$) if it is frequent and no *strict* superset of it is frequent.

Definition 1.7 (Frequent/infrequent border). The search space of all itemsets (the subset lattice of $\mathcal{U}$) can be partitioned into frequent and infrequent itemsets. The *border* is the boundary between the two regions; by downward closure the frequent region is “upward-closed” downward from the top, i.e. every subset of a frequent itemset is frequent, and every superset of an infrequent itemset is infrequent. The maximal frequent itemsets lie on this border.

Example 1.3 (Maximal itemsets of the grocery database). At $minsup = 2$ the frequent itemsets are (Section 3 derives all 11): the 1-itemsets {milk} (5), {eggs} (3), {yogurt} (3), {bread} (2), {cheese} (2); the 2-itemsets {eggs,milk} (3), {milk,yogurt} (3), {bread,milk} (2), {cheese,milk} (2), {eggs,yogurt} (2); and the single 3-itemset {eggs,milk,yogurt} (2). The maximal frequent itemsets are exactly the three that have no frequent superset:

$$\{\text{bread, milk}\}, \quad \{\text{cheese, milk}\}, \quad \{\text{eggs, milk, yogurt}\}.$$

For example {bread,milk} is maximal because its only strict supersets {bread,butter,milk} and {bread,cheese,milk} have support $1 < 2$. The 1-itemset {bread} is *not* maximal because {bread,milk} is a frequent superset. The notebook asserts 3 maximal itemsets with `maxfi_apriori[0] == (("eggs","milk","yogurt"), 2)`.

Counting itemsets quickly

The number of distinct k -itemsets over d items is the binomial coefficient $\binom{d}{k}$, and the total number of *non-empty* itemsets is $2^d - 1$. For $d = 6$ that is 63; for $d = 1000$ it is about $2^{1000} \approx 10^{301}$, a number with 302 digits. This exponential blow-up is the whole reason FIM needs smarter algorithms than brute force.

Chapter 2

Brute-Force Enumeration and Scalability

Learning objectives

After this chapter you should be able to:

- describe the brute-force algorithm;
- state and justify its $O(N \cdot 2^d)$ time complexity;
- predict how runtime changes when d grows (exponential) vs. when N grows (linear);
- map the notebook's `brute` and `to_sparse` functions to a manual procedure.

2.1 The brute-force algorithm

The most direct approach is: enumerate *every* non-empty subset of $\mathcal{U}$, compute its support by scanning all N transactions, and report those with $\text{sup}(X) \geq \text{minsup}$.

Algorithm 2.1 (Brute force). 1. Let $\mathcal{U} = \bigcup_i T_i$ (all distinct items), and set `frequent` = [].

2. For each size $k = 1, 2, \dots, d$: for each k -itemset $X \subseteq \mathcal{U}$ (via `itertools.combinations`):

3. compute $\text{sup}(X) = \sum_{T_i \in \mathcal{T}} \mathbf{1}[X \subseteq T_i]$;

4. if $\text{sup}(X) \geq \text{minsup}$, record $(X, \text{sup}(X))$.

5. Sort the records by (*decreasing support, decreasing size, alphabetical order*) and return them.

The sort key in Python is the tuple `(-support, -len(itemset), itemset)`, which sorts largest support first, breaks ties by larger itemsets first, and further ties alphabetically. This is the exact convention the notebooks use, so reproduce it in any manual trace.

2.2 Complexity analysis

At level k there are $\binom{d}{k}$ candidates, and each candidate requires one full pass over N transactions (with a subset test whose cost is up to $O(k) \subseteq O(d)$). Summing over all k :

$$\sum_{k=1}^d \binom{d}{k} = 2^d - 1$$

candidates, each scanned against N transactions, giving overall time

$$O(N \cdot 2^d),$$

or $O(N \cdot d \cdot 2^d)$ if the per-transaction subset test is charged at $O(d)$.

Example 2.1 (Scaling prediction — Problem 4). The notebook asks for the runtime ratio in two scenarios, starting from the current case $N = 5$, $d = 6$ (which generates $2^6 - 1 = 63$ itemsets, $5 \times 63 = 315$ subset checks):

- **d grows to 1000, N fixed at 5:** candidates grow from 63 to $2^{1000} - 1$, so the ratio is $\frac{2^{1000} - 1}{2^6 - 1} \approx 1.7 \times 10^{299}$. The algorithm is effectively *impossible*: this is the exponential blow-up in the number of items.
- **N grows to 1000, d fixed at 6:** the number of candidates stays 63; only the scan length grows, so the ratio is $1000/5 = 200$, i.e. *linear* in N .

Takeaway: FIM's fundamental difficulty is the exponential number of itemsets as the number of distinct items grows, not the (merely linear) cost of more transactions.

Charging the subset test

The notebook's group-discussion note gives $O(N \times 2^d)$ and remarks that if checking a subset costs time proportional to its size the bound becomes $O(N \times d \times 2^d)$. Both are standard; the essential point to reproduce on an exam is the *exponential in d , linear in N* structure. This supplement only makes explicit a small ambiguity (whether the $O(d)$ factor is counted) that the slides leave implicit.

Chapter 3

The Apriori Algorithm

Learning objectives

After this chapter you should be able to:

- explain why the search space is a lattice and how the border partitions it;
- state the candidate-generation and pruning steps;
- execute Apriori by hand level by level on a small database;
- explain the maximal-itemset variant `apriori2`.

3.1 Idea: breadth-first search with pruning

Brute force wastes enormous effort enumerating itemsets that cannot possibly be frequent. The key realization is that finding frequent itemsets is a *search* over the subset lattice. Figure 1 of the slides shows this lattice for 5 items: layer k holds the k -itemsets, and an edge connects a k -itemset to a $(k + 1)$ -itemset obtained by adding one item.

The *border* (blue broken line in the figure) separates frequent from infrequent itemsets. Because of downward closure (Corollary 1.2), the region is “closed upward-downward”: if a node is infrequent, everything above it (all supersets) is infrequent too. Apriori therefore performs a *breadth-first search from the bottom*: it checks 1-itemsets, keeps only frequent ones, uses them to generate candidate 2-itemsets, prunes, checks, and so on, never descending past the border.

3.2 The three building blocks

The notebook implements Apriori with two helpers plus the main loop.

Algorithm 3.1 (Candidate generation — `candidate_generation`). Given the frequent $(k - 1)$ -itemsets F_{k-1} and size k :

1. Collect every item that appears in *any* itemset of F_{k-1} .
2. Form all $\binom{|\text{items}|}{k}$ combinations of those items.

This is deliberately *superset* generation: it may produce candidates that have infrequent subsets; the next step removes them.

Candidate generation — time

Let m be the number of distinct items appearing in F_{k-1} (so $m \leq d$). This step materializes $\binom{m}{k}$ combinations of size k , each costing $O(k)$ to build, giving $O(\binom{m}{k} \cdot k) \subseteq O(\binom{d}{k} \cdot k)$. This is the step that can explode: even if pruning later discards most candidates, they are first generated, and this generation cost is what sets Apriori's worst-case bound.

Algorithm 3.2 (Pruning — `remove_candidate`). Given candidate set C_k , size k , and F_{k-1} :

1. For each candidate $c \in C_k$, generate all its $(k-1)$ -subsets.
2. Keep c only if *every* $(k-1)$ -subset of c is in F_{k-1} .
3. Discard any candidate with at least one infrequent subset.

By downward closure, a candidate with an infrequent subset cannot itself be frequent, so this pruning is *safe*: it never removes a truly frequent itemset.

Pruning — time

For each of the $|C_k|$ candidates, generate its k subsets of size $k-1$ and test membership in F_{k-1} . Building one subset costs $O(k)$ and there are k of them, so $O(|C_k| \cdot k^2)$ in the naive implementation; with a hash set of F_{k-1} the membership test is $O(1)$ after $O(k)$ to build each subset, i.e. $O(|C_k| \cdot k)$ in practice. Either way it is linear in the number of candidates and polynomial in k — cheap relative to the database scan that follows.

- Algorithm 3.3** (Apriori). 1. Scan $\mathcal{T}$ once, count each single item, and form $F_1 = \{\{i\} : \text{sup}(\{i\}) \geq \text{minsup}\}$.
2. For $k = 2, 3, \dots$ while $F_{k-1} \neq \emptyset$:
 3. $C_k \leftarrow \text{candidate_generation}(F_{k-1}, k)$;
 4. $C_k \leftarrow \text{remove_candidate}(C_k, k, F_{k-1})$;
 5. scan $\mathcal{T}$ to compute $\text{sup}(c)$ for each $c \in C_k$;
 6. $F_k \leftarrow \{c \in C_k : \text{sup}(c) \geq \text{minsup}\}$.
 7. Return all F_k merged, sorted by (decreasing support, decreasing size, alphabetical).

Apriori — time and memory

Apriori scans $\mathcal{T}$ once to build F_1 , then once per subsequent level, so the number of scans is $O(L)$ where L is the length of the longest frequent itemset. At level k the scan costs $O(N \cdot |C_k| \cdot k)$: $|C_k|$ candidates, each subset-tested against N transactions in $O(k)$. Summing over levels,

$$O\left(\sum_{k=1}^L N |C_k| k\right).$$

In the worst case $|C_k| = \binom{d}{k}$ (no pruning), which is again $O(N \cdot d \cdot 2^{d-1})$. The practical saving is that downward closure keeps $|C_k| \leq |F_k|$, and the number of

frequent itemsets is usually far smaller than 2^d . Memory is $O(\sum_k |F_k|)$ — only the current and found itemsets are kept, not the whole lattice.

3.3 Fully worked manual trace

We run Apriori on the grocery database with $\text{minsup} = 2$.

Level $k = 1$. Item counts: milk 5, eggs 3, yogurt 3, bread 2, cheese 2, butter 1. Only butter fails, so

$$F_1 = \{\{\text{bread}\}, \{\text{cheese}\}, \{\text{eggs}\}, \{\text{milk}\}, \{\text{yogurt}\}\}.$$

Level $k = 2$. Candidate generation takes the 5 distinct items and forms all $\binom{5}{2} = 10$ pairs:

$$C_2 = \{\text{bc}, \text{be}, \text{bm}, \text{by}, \text{ce}, \text{cm}, \text{cy}, \text{em}, \text{ey}, \text{my}\}$$

(where b=bread, c=cheese, e=eggs, m=milk, y=yogurt). Since every single item here is frequent, *no* candidate is pruned. Counting support:

candidate	be	bm	ce	cm	em	ey	my	bc	by	cy
sup	1	2	1	2	3	2	3	1	0	1

Keeping $\text{sup} \geq 2$:

$$F_2 = \{\text{bm}(2), \text{cm}(2), \text{em}(3), \text{ey}(2), \text{my}(3)\}.$$

Level $k = 3$. The items present in F_2 are $\{\text{b}, \text{c}, \text{e}, \text{m}, \text{y}\}$, giving $\binom{5}{3} = 10$ candidate triples. Pruning removes every triple with an infrequent pair (recall the only frequent pairs are bm, cm, em, ey, my):

- bcm: has bc (infreq) $\Rightarrow$ prune. bce: bc infreq $\Rightarrow$ prune. bcy: bc, by, cy infreq $\Rightarrow$ prune.
- bme: all of bm, be, em? be is infrequent $\Rightarrow$ prune.
- bmy: by infreq $\Rightarrow$ prune. bey: be, by infreq $\Rightarrow$ prune.
- cme: ce infreq $\Rightarrow$ prune. cmy: cy infreq $\Rightarrow$ prune. cey: ce, cy infreq $\Rightarrow$ prune.
- emy: subsets em, ey, my are *all* frequent $\Rightarrow$ **keep**.

Only **emy** survives: $C_3 = \{\text{emy}\}$. Its support is 2 (transactions T_2 and T_4), so $F_3 = \{\text{emy}(2)\}$.

Level $k = 4$. F_3 contains a single itemset; candidate generation can form $\binom{3}{4} = 0$ quadruples, and $F_4 = \emptyset$, so the loop stops.

Result. The 11 frequent itemsets, sorted by (decreasing support, decreasing size, alphabetical), begin with (milk, 5) and are exactly the list given in Example 1.3. The notebook asserts `len(fi_apriori)==11` and `fi_apriori[0]==(("milk",),5)`.

Pruning is where the marks are

When you hand-trace Apriori, most of the “work” is at the $k = 3$ level: write out the candidates, then *explicitly* cross off each one that contains an infrequent $(k - 1)$ -subset. Examiners look for (i) correct F_1 , (ii) correct C_k , (iii) a clear note of *which subset* killed each pruned candidate, and (iv) correct support counts for the survivors.

3.4 The maximal-itemset variant `apriori2`

Algorithm 3.4 (Apriori for maximal itemsets — `apriori2`). Run Apriori exactly as in Algorithm 3.3 to collect *all* frequent itemsets with their supports. Then, for each frequent itemset X , mark it maximal if there is no other frequent itemset Y with $X \subsetneq Y$; discard the rest. Sort as usual.

Maximal-itemset test — time

The added maximality test compares every frequent itemset against every other. Letting $|F| = \sum_k |F_k|$ be the total number of frequent itemsets and d the number of items, a naive nested loop costs $O(|F|^2 \cdot d)$: $O(|F|^2)$ pairs, each subset test $O(d)$. The Apriori pass that precedes it still dominates with $O(\sum_k N|C_k|k)$, so `apriori2` inherits Apriori’s worst-case bound and only appends this quadratic-in- $|F|$ cleanup.

The test `itemset < other` in Python is set *proper*-subset comparison, so an itemset survives only if no frequent itemset strictly contains it. On the grocery database this returns the three maximal itemsets of Example 1.3, sorted so that (eggs, milk, yogurt) (support 2) comes first.

Maximal vs. closed

The slides mention only *maximal* frequent itemsets. A related notion you may encounter elsewhere is the *closed* itemset: an itemset X is closed if no strict superset of X has the *same* support. Every maximal itemset is closed, but not conversely. This is supplementary context, not required by the course material, but it clarifies why some references emphasize “closed patterns.”

Chapter 4

The FP-Growth Algorithm

Learning objectives

After this chapter you should be able to:

- explain the prefix-tree (enumeration-tree) idea and projected databases;
- build an FP-tree from a transaction database;
- construct the header table and node links;
- form conditional pattern bases and conditional trees;
- execute the recursive `mine_tree` and recover all frequent itemsets;
- compare FP-Growth with Apriori (time and memory).

4.1 Idea: compress the database into a tree

Apriori's weakness is that it generates *candidate* itemsets that may not appear in the data at all, and it re-scans the database once per level. *FP-Growth* is a *pattern-growth* algorithm that avoids explicit candidate generation by compressing the database into a *frequent-pattern tree* (FP-tree) and then mining that tree recursively.

Definition 4.1 (Prefix tree). Assume a fixed order on items (say alphabetical). A *prefix tree* is a tree where each node is an item and an ancestor always precedes a descendant in the order. Each root-to-node path is an itemset; shared prefixes are stored once, which is what makes the FP-tree compact.

Definition 4.2 (Projected database). For a prefix (an itemset that is a frequent “prefix” of larger patterns), the *projected database* is the multiset of transactions containing that prefix, with the prefix items *removed*. FP-Growth mines recursively: it finds frequent single items in the projected database and recurses.

Example 4.1 (Projected database for bread). At $minsup = 2$, bread is frequent ($sup = 2$), appearing in T_1 and T_3 . Removing the prefix {bread}:

$$T_1 \rightarrow \{\text{butter, milk}\}, \quad T_3 \rightarrow \{\text{cheese, eggs, milk}\}.$$

Scanning this projected database, only milk has support ≥ 2 (butter 1, cheese 1, eggs 1), so FP-Growth records {bread, milk} as frequent and descends no further (the next projection would be empty).

4.2 The FP-tree, header table, and node links

Algorithm 4.1 (FP-tree construction). 1. Scan $\mathcal{T}$ once to count each item; keep only items with $\text{sup} \geq \text{minsup}$ (“frequent items”), recording their counts.

2. Order frequent items by *decreasing support*, breaking ties alphabetically; assign ranks $0, 1, 2, \dots$ in that order.

3. For each transaction, keep only its frequent items and sort them by that rank.

4. Insert each ordered transaction into the tree from the root: follow the existing path as long as items match (incrementing counts), and create new nodes for the remainder. For every new node of item i , append it to item i ’s *node link* chain in the header table.

FP-tree construction — time and memory

Two scans of $\mathcal{T}$ are needed (one to count, one to insert). Sorting each transaction by rank costs $O(|T_i| \log |T_i|)$ for transaction T_i ; summing over all transactions gives $O(\sum_i |T_i| \log |T_i|)$, i.e. $O(Nt \log t)$ with t the average transaction length. Each insertion touches $O(|T_i|)$ nodes, adding another $O(\sum_i |T_i|)$. The whole construction is thus linear in the database size up to a logarithmic sort factor — the key contrast with Apriori, which re-scans once per level. Memory is $O(\#nodes)$, the size of the compressed tree, which can be much smaller than N when prefixes are shared but can approach $O(\sum_i |T_i|)$ when they are not.

The *header table* stores, for each frequent item, a triple [support, head of node links, tail of node links]. The node links thread together every tree node carrying the same item, which is how FP-Growth can later find all occurrences of an item without searching the whole tree.

4.3 Fully worked manual trace of FP-Growth

Step 1 — frequent items and order. Item counts: milk 5, eggs 3, yogurt 3, bread 2, cheese 2; butter 1 is dropped. Order by (decreasing support, alphabetical):

$$\underbrace{\text{milk}}_5 > \underbrace{\text{eggs, yogurt}}_3 > \underbrace{\text{bread, cheese}}_2.$$

So the rank order is milk(0), eggs(1), yogurt(2), bread(3), cheese(4).

Step 2 — sort each transaction.

T_i	sorted (frequent items only)
T_1	milk, bread
T_2	milk, eggs, yogurt
T_3	milk, eggs, bread, cheese
T_4	milk, eggs, yogurt
T_5	milk, yogurt, cheese

Step 3 — build the tree. Inserting each path (incrementing shared prefixes) yields:

insert	effect
(milk, bread)	milk:1 → bread:1
(milk, eggs, yogurt)	milk:2 → eggs:1 → yogurt:1
(milk, eggs, bread, cheese)	milk:3 → eggs:2 → bread:1 → cheese:1
(milk, eggs, yogurt)	milk:4 → eggs:3 → yogurt:2
(milk, yogurt, cheese)	milk:5 → yogurt:1 → cheese:1

The header table records the support of each item (milk 5, eggs 3, yogurt 3, bread 2, cheese 2) and threads its node links.

Step 4 — mine the tree (ascending support order). `mine_tree` processes items in *ascending* support order: cheese, bread, yogurt, eggs, milk. For each item it (a) records the itemset `prefix + (item,)` with the item's support, (b) builds that item's *conditional pattern base* (all prefix paths to that item, with counts), and (c) recursively mines the resulting conditional tree.

- **cheese** (2): node links lead to two cheese nodes — one under (milk,eggs,bread) with count 1 and one under (milk,yogurt) with count 1. Pattern base: {milk, eggs, bread : 1}, {milk, yogurt : 1}. In the conditional tree only milk reaches support 2, so we record {cheese} and {cheese, milk} (each 2).
- **bread** (2): pattern base {milk : 1}, {milk, eggs : 1}; only milk survives; record {bread}, {bread, milk} (each 2).
- **yogurt** (3): pattern base {milk, eggs : 2}, {milk : 1}. Both milk (3) and eggs (2) survive. Record {yogurt} (3). Recursing on the conditional tree: *eggs* gives {yogurt, eggs} (2), then {yogurt, eggs, milk} (2); *milk* gives {yogurt, milk} (3).
- **eggs** (3): pattern base {milk : 3}; milk survives; record {eggs} (3), {eggs, milk} (3).
- **milk** (5): pattern base is empty (milk is a root child); record {milk} (5).

The union, deduplicated and sorted, is again the 11 frequent itemsets; the notebook asserts `len(fi_fpgrowth)==11` and `fi_fpgrowth[0]==(("milk",),5)`.

Ascending support order

The recursion processes items from *smallest* support to *largest* (`sorted(..., key=lambda item: (support, item))`). This is a common trap: people wrongly expect descending order (which is used for *sorting the path* in the tree, not for *mining*). Remember: *path order descending, mine order ascending*.

FP-Growth mining — time

Once the FP-tree is built, mining needs *no further database scans*: all work happens in memory over the tree and its conditional trees. In the worst case the cost is proportional to the number of frequent itemsets times the number of tree nodes, $O(\# \text{frequent itemsets} \times \# \text{nodes})$, because each frequent itemset is materialized once and each recursion walks the node-link chains of its conditional pattern base. When the tree stays compact (shared prefixes), this is far below the candidate space of

Apriori; the bound degrades toward the worst case only when patterns share little prefix structure.

4.4 Apriori vs. FP-Growth

	Apriori	FP-Growth
Strategy	breadth-first, generate-and-test	divide-and-conquer over projected databases
Candidate generation	explicit, level by level	<i>no</i> explicit candidates
Database scans	one per level ($O(\text{max itemset size})$ scans)	two passes to build the tree, then mine in memory
Time (many items)	slow: candidate space explodes with d	faster: prunes via compressed tree
Time (many transactions)	re-scans repeatedly	faster: few scans before mining
Memory	low (only current level)	<i>higher</i> : FP-tree must fit in memory

Why FP-Growth wins

FP-Growth stores the database's frequent patterns in a *compressed tree* and never materializes candidate itemsets that are absent from the data. For large d or large N this eliminates the dominant cost of Apriori. The price is memory: the FP-tree (and its conditional trees) must reside in RAM, which becomes the bottleneck for very large datasets.

Chapter 5

Association Rule Mining

Learning objectives

After this chapter you should be able to:

- define association rule, antecedent, consequent, confidence, and lift;
- interpret $\text{lift} > 1$, $= 1$, < 1 ;
- generate single-item-consequent rules from frequent itemsets;
- sort rules by (decreasing lift, decreasing confidence, decreasing support);
- reproduce the 13-rule manual trace of the notebook.

5.1 Definitions

Frequent itemset mining usually continues into *association rule mining*.

Definition 5.1 (Association rule). An *association rule* is an implication $A \rightarrow B$ where A (the *antecedent*) and B (the *consequent*) are disjoint itemsets.

Definition 5.2 (Confidence). The *confidence* of $A \rightarrow B$ is the conditional probability of finding B given A ,

$$\text{conf}(A \rightarrow B) = \Pr(B \in T_i \mid A \in T_i) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}.$$

Definition 5.3 (Lift). The *lift* of $A \rightarrow B$ measures how much more likely B becomes once we know A is present,

$$\text{lift}(A \rightarrow B) = \frac{\text{conf}(A \rightarrow B)}{\text{relSup}(B)}.$$

Reading lift

- $\text{lift} > 1$: A and B co-occur *more* often than chance — positive association (useful for recommendation).
- $\text{lift} = 1$: no association — A and B are independent.

- lift < 1: negative association — A and B repel each other.

Lift is symmetric in a sense confidence is not: it normalizes out the baseline popularity of B . A rule with high confidence but lift ≈ 1 merely reflects that B is common anyway, and is not informative.

Confidence vs. lift

High confidence alone is *not* evidence of a useful rule. If B appears in 90% of transactions, then even a random A gives $\text{conf}(A \rightarrow B) \approx 0.9$. Lift divides by $\text{relSup}(B)$, the baseline, and is the correct measure of *interestingness* — an exam question will often ask you to explain why a high-confidence rule can still have lift ≤ 1 .

5.2 Generating rules

The notebook's `assoc` restricts attention to rules whose *consequent* is a *single item* and whose antecedent is all remaining items in the frequent itemset (this keeps the number of rules manageable; general ARM also considers multi-item consequents). For each frequent itemset X with $|X| \geq 2$ and each item $i \in X$:

1. let consequent = $\{i\}$, antecedent = $X \setminus \{i\}$;
2. if the antecedent or the consequent is missing from the support table, skip;
3. $\text{conf} = \frac{\text{sup}(X)}{\text{sup}(\text{antecedent})}$; discard if $\text{conf} < \text{minconf}$;
4. $\text{lift} = \frac{\text{conf}}{\text{relSup}(\text{consequent})} = \frac{\text{conf}}{\text{sup}(\{i\})/N}$.

Finally, sort rules by (*decreasing lift, decreasing confidence, decreasing support*), keeping Python's stable order for exact ties.

5.3 Fully worked manual trace (the 13 rules)

The notebook calls `assoc` on the following frequent itemsets with $N = 5$ and $\text{minconf} = 0.2$:

(a, 5), (a, b, 3), (a, c, 3), (b, 3), (c, 3), (a, b, c, 2), (a, d, 2), (a, e, 2), (b, c, 2), (d, 2), (e, 2).

So $\text{sup}(a) = 5$, $\text{sup}(b) = 3$, $\text{sup}(c) = 3$, $\text{sup}(d) = 2$, $\text{sup}(e) = 2$. Every candidate rule has confidence ≥ 0.2 , so none is discarded. Enumerate the rules and compute lift (recall $N = 5$, so $\text{relSup}(a) = 1$, $\text{relSup}(b) = 0.6$, $\text{relSup}(c) = 0.6$, $\text{relSup}(d) = 0.4$, $\text{relSup}(e) = 0.4$):

rule	from	conf	lift	supp
$b \rightarrow a$	ab	$3/3 = 1.0$	$1.0/1.0 = 1.0$	3
$a \rightarrow b$	ab	$3/5 = 0.6$	$0.6/0.6 = 1.0$	3
$c \rightarrow a$	ac	$3/3 = 1.0$	$1.0/1.0 = 1.0$	3
$a \rightarrow c$	ac	$3/5 = 0.6$	$0.6/0.6 = 1.0$	3
$bc \rightarrow a$	abc	$2/2 = 1.0$	$1.0/1.0 = 1.0$	2
$ac \rightarrow b$	abc	$2/3 = 0.667$	$0.667/0.6 = \mathbf{1.111}$	2
$ab \rightarrow c$	abc	$2/3 = 0.667$	$0.667/0.6 = \mathbf{1.111}$	2
$d \rightarrow a$	ad	$2/2 = 1.0$	$1.0/1.0 = 1.0$	2
$a \rightarrow d$	ad	$2/5 = 0.4$	$0.4/0.4 = 1.0$	2
$e \rightarrow a$	ae	$2/2 = 1.0$	$1.0/1.0 = 1.0$	2
$a \rightarrow e$	ae	$2/5 = 0.4$	$0.4/0.4 = 1.0$	2
$c \rightarrow b$	bc	$2/3 = 0.667$	$0.667/0.6 = \mathbf{1.111}$	2
$b \rightarrow c$	bc	$2/3 = 0.667$	$0.667/0.6 = \mathbf{1.111}$	2

There are 13 rules in total, matching the notebook assertion `len(rules)==13`. Sorting by (decreasing lift, decreasing confidence, decreasing support) with stable tie-breaking puts the four lift-1.111 rules first; the very first rule is $ac \rightarrow b$ with confidence 0.667 and lift 1.111, exactly the notebook's assertion.

The stable-sort subtlety

The four rules with lift 1.111 all have confidence 0.667 and support 2, so their order is determined by *insertion order* (Python's stable sort). They are emitted in the order $ac \rightarrow b$, $ab \rightarrow c$, $c \rightarrow b$, $b \rightarrow c$. If a question asks for "the first rule," reproduce *this* order; do not alphabetize the ties.

Chapter 6

The pyFIM Library and the Online Retail Case Study

Learning objectives

After this chapter you should be able to:

- use `fim.apriori`, `fim.eclat`, and `fim.fpgrowth`;
- explain the `supp` sign convention (positive = percentage, negative = absolute);
- use `zmin`, `target`, `conf`, and `report`;
- interpret the Online Retail output and produce business recommendations.

6.1 Using pyFIM

Pandas and scikit-learn do not ship a frequent-itemset module; the course uses [pyFIM](#). The core calls are:

- `fim.apriori(db)` — Apriori over the transaction list;
- `fim.eclat(db, supp=-2)` — the Eclat (equivalence-class, depth-first) algorithm with *absolute* support 2;
- `fim.fpgrowth(db, supp=20, zmin=2)` — FP-Growth with support 20% and minimum itemset length 2;
- `fim.fpgrowth(db, target="r", supp=20, conf=30, report="1")` — association rules (`target="r"`) with 20% support, 30% confidence, reporting *lift* (`report="1"`).

The `supp` sign convention

A *positive* `supp` value is interpreted as a **percentage** (relative support); a *negative* value is interpreted as an **absolute** count. Thus `supp=-2` means “support at least 2 transactions,” while `supp=20` means “at least 20% of transactions.”

Reading the rule output

With `target="r"` each returned row is a triple (`consequent`, (`antecedent`,...), `count`). For example (`'JUMBO BAG RED RETROSPOT'`, (`'DOTCOM POSTAGE'`,), 485) means the rule `DOTCOM POSTAGE → JUMBO BAG RED RETROSPOT` holds in 485 transactions. The consequent is the single item *first*; the antecedent items are in a tuple *second*.

6.2 Online Retail case study

The notebook loads `Online Retail.xlsx`, keeps only `InvoiceNo` and `Description`, then:

1. casts `InvoiceNo` to string;
2. **drops cancelled invoices** (those starting with "C") and rows with no description;
3. removes duplicate products within an invoice (`drop_duplicates`);
4. groups by `InvoiceNo` to form one basket (list) per invoice.

Example 6.1 (Dataset statistics). The cleaned dataset has 20,610 transactions, 4,207 unique products, and a largest basket of 1,108 items. FP-Growth is run with `supp=-100` (at least 100 transactions), `zmin=2`, and `target="s"`.

The top itemset is (`'DOTCOM POSTAGE'`, `'JUMBO BAG RED RETROSPOT'`) appearing in 485 transactions; the trio (`'DOTCOM POSTAGE'`, `'WHITE HANGING HEART T-LIGHT HOLDER'`, `'JUMBO BAG RED RETROSPOT'`) appears in 199 transactions. Association rules are mined with `supp=-100`, `conf=50`, `zmin=2`, `target="r"`.

Example 6.2 (Business recommendations). The notebook derives three suggestions for the store owner:

1. **Bundle** *JUMBO BAG RED RETROSPOT* with *WHITE HANGING HEART T-LIGHT HOLDER* (449 co-occurring transactions) for promotions or recommendations.
2. **Promote** *PACK OF 72 RETROSPOT CAKE CASES* with *REGENCY CAKE-STAND 3 TIER* (358 co-occurrences), a strong cross-sell pair.
3. Add a “**Frequently Bought Together**” section using these combinations to nudge basket size upward.

It also cautions that *DOTCOM POSTAGE* should be treated separately: it is postage (a shipping artifact), not merchandise, so it is less useful for bundling or merchandising decisions.

Chapter 7

Summary: Formulas, Algorithms, and Exam Strategy

7.1 Formula sheet (Part I)

Absolute and relative support

$\text{sup}(X) = |\{T_i \in \mathcal{T} : X \subseteq T_i\}|$, $\text{relSup}(X) = \frac{\text{sup}(X)}{N}$. X is frequent iff $\text{sup}(X) \geq \text{minsup}$.

Support monotonicity / downward closure

$Y \subseteq X \implies \text{sup}(Y) \geq \text{sup}(X)$. Hence every subset of a frequent itemset is frequent; every superset of an infrequent itemset is infrequent.

Confidence and lift

$\text{conf}(A \rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}$, $\text{lift}(A \rightarrow B) = \frac{\text{conf}(A \rightarrow B)}{\text{relSup}(B)}$. Lift > 1 : positive association; $= 1$: independence; < 1 : negative association.

Complexities

Brute force: $O(N \cdot 2^d)$. Apriori: exponential in d in the worst case but pruned by downward closure. FP-Growth: two database scans + in-memory tree mining; no explicit candidates but higher memory.

7.2 Algorithm summaries

- **Brute force** — enumerate all $2^d - 1$ itemsets, count each, keep frequent. Simple, exact, infeasible for large d .
- **Apriori** — level-wise: F_1 from item counts; then $C_k = \text{candidate_generation}(F_{k-1})$, prune with `remove_candidate`, count, keep frequent. Stops when a level is empty.
- **FP-Growth** — count items, order by decreasing support, build FP-tree with node links; recursively mine conditional pattern bases/trees in ascending support order.

- **Association rules** — from frequent itemsets, single-item consequents; filter by *minconf*; rank by (lift, confidence, support).

7.3 Decision guide

- Asked to *find all frequent itemsets on a tiny database by hand*? Use Apriori (easiest to show level-by-level) or FP-Growth if the question says so.
- Asked to *find only the maximal itemsets*? Run Apriori then keep itemsets with no frequent strict superset.
- Asked to *explain why we don't use brute force*? Cite the $O(N \cdot 2^d)$ blow-up (Example 2.1).
- Asked to *compare Apriori and FP-Growth*? Time vs. memory trade-off (Section above).
- Asked to *judge whether a rule is interesting*? Compute lift, not just confidence.
- Asked to *read pyFIM output*? Recall the **supp** sign convention and the (consequent, antecedent, count) tuple.

7.4 Common exam traps and time savers

Most common mistakes

- Mixing up *absolute* and *relative* support (and the **supp** sign in pyFIM).
- Forgetting that a k -itemset has $\binom{k}{k-1} = k$ subsets to check in **remove_candidate** — check *all* of them.
- Pruning a candidate that is actually frequent, by only testing *some* of its subsets.
- Confusing the tree's *descending* path order with the *ascending* mining order in FP-Growth.
- Reporting high-confidence rules as “interesting” without checking lift.
- Including infrequent items (e.g. butter, support 1) in FP-Growth paths — they must be removed first.

Time savers

- To check frequency of a candidate, list the transactions that contain it; count them once, carefully.
- At level k , only *items that appear in F_{k-1}* can generate candidates — this prunes many triples immediately.
- For lift, remember $\text{relSup}(B) = \text{sup}(B)/N$; if you already know $\text{sup}(B)$ and N you can skip a division.
- A quick sanity check on support: the maximum possible support of any itemset is sup of its rarest item; if your count exceeds that, you erred.

Prerequisite reminders

- Set notation: $Y \subseteq X$ means Y is a subset (may equal X); $Y \subsetneq X$ is a *proper* subset. Monotonicity uses $\subseteq$; maximality uses $\subsetneq$.
- Binomial coefficients: the number of k -subsets of a d -set is $\binom{d}{k}$.
- Sum of a full binomial row: $\sum_{k=0}^d \binom{d}{k} = 2^d$.

Chapter 8

Practice Problems — Part I

Solutions and point-based rubrics are in Part IV (the answer key). Attempt every problem before consulting it.

8.1 Category A — Conceptual

Exercise 8.1 (Support and relative support). Category A — Conceptual Easy, 2 min Define absolute support and relative support of an itemset, and give the conversion between them. For the grocery database, compute $\text{relSup}(\{\text{bread}, \text{milk}\})$.

Exercise 8.2 (Monotonicity and pruning). Category A — Conceptual Easy, 3 min State the support-monotonicity (downward-closure) property, and explain in one or two sentences *why* it is safe for Apriori to discard a candidate whose subset is infrequent.

Exercise 8.3 (Frequent vs. maximal). Category A — Conceptual Moderate, 4 min Distinguish a frequent itemset from a maximal frequent itemset. Can a maximal frequent itemset contain a frequent proper subset? Can a frequent itemset have a frequent proper *superset* and still be maximal? Justify.

Exercise 8.4 (Lift interpretation). Category A — Conceptual Easy, 3 min Define lift and explain what $\text{lift}(A \rightarrow B) > 1$, $= 1$, and < 1 each mean. Why is lift preferred over confidence for judging whether a rule is interesting?

8.2 Category B — Basic computation

Exercise 8.5 (Support computations). Category B — Basic computation Easy, 4 min Using the grocery database, compute by hand: $\text{sup}(\{\text{eggs}, \text{milk}\})$, $\text{sup}(\{\text{bread}, \text{cheese}\})$, $\text{relSup}(\{\text{milk}\})$, and $\text{sup}(\{\text{eggs}, \text{milk}, \text{yogurt}\})$.

Exercise 8.6 (Confidence and lift). Category B — Basic computation Moderate, 5 min Suppose $N = 20$, $\text{sup}(A) = 10$, $\text{sup}(B) = 8$, $\text{sup}(A \cup B) = 4$. Compute $\text{relSup}(B)$, $\text{conf}(A \rightarrow B)$, and $\text{lift}(A \rightarrow B)$, and interpret the lift.

8.3 Category C — Manual algorithm execution

Exercise 8.7 (Apriori by hand). Category C — Manual algorithm execution Moderate, 10 min Consider the database

$$T_1 = \{a, b, c\}, T_2 = \{a, b, d\}, T_3 = \{a, c\}, T_4 = \{b, c, d\}, T_5 = \{a, b\}$$

with $\text{minsup} = 2$. Run Apriori by hand: report F_1 , C_2 , F_2 , C_3 (after pruning), F_3 , and hence *all* frequent itemsets with their supports. Which are maximal?

Exercise 8.8 (Association rules by hand). Category C — Manual algorithm execution Moderate, 8 min For the database of Exercise 8.7 (frequent itemsets at $\text{minsup} = 2$), generate single-item-consequent association rules with $\text{minconf} = 0.5$, computing confidence and lift for each. Report the top two rules by (decreasing lift, decreasing confidence, decreasing support).

Exercise 8.9 (FP-Growth by hand). Category C — Manual algorithm execution Difficult, 12 min For the grocery database at $\text{minsup} = 2$: (a) list the frequent items in decreasing support order, (b) build the FP-tree after sorting each transaction, (c) state the header table, and (d) list every itemset the ascending-support recursion produces, with its support.

8.4 Category D — Multi-step / integrated

Exercise 8.10 (Full FIM $\rightarrow$ ARM pipeline). Category D — Multi-step / integrated Difficult, 15 min Using the database of Exercise 8.7 at $\text{minsup} = 2$: find all frequent itemsets, then all association rules with $\text{minconf} = 0.5$, compute confidence and lift for each rule, sort them, and write a one-sentence business interpretation of the strongest rule.

8.5 Category E — Challenging

Exercise 8.11 (Prove monotonicity). Category E — Challenging Moderate, 6 min Prove that if $Y \subseteq X$ then $\text{sup}(Y) \geq \text{sup}(X)$. Then use it to justify the downward-closure property.

Exercise 8.12 (Worst-case analysis). Category E — Challenging Difficult, 10 min Explain why Apriori's *worst-case* number of candidates is still $2^d - 1$, even though it prunes. Give a concrete (small) database and minsup where Apriori must enumerate every non-empty itemset. When is the pruning most effective, and when is it useless?

Exercise 8.13 (Choosing thresholds). Category E — Challenging Moderate, 8 min In the Online Retail case study, $\text{minsup} = 100$ transactions and $\text{conf} = 50\%$ were chosen. Explain (i) what happens to the number of discovered itemsets and rules if minsup is lowered, and (ii) why DOTCOM POSTAGE dominates the top rules yet is excluded from merchandising recommendations.

Part II

Neighborhood-based Collaborative Filtering

Part II — Topic Checklist

Tick each item once you can (a) state the definition from memory, (b) explain the intuition in your own words, and (c) carry out a small manual example.

- ☐ Utility matrix: users, items, ratings (utility), and the sparsity of real matrices.
- ☐ Explicit vs. implicit ratings (and how to convert one to the other).
- ☐ Relevant / novel / serendipitous / diverse recommendations.
- ☐ Content-based vs. collaborative filtering; memory-based vs. model-based.
- ☐ Raw cosine similarity over common items, and when it is 0.
- ☐ Adjusted cosine = Pearson correlation; why we mean-center first.
- ☐ User-based CF: peer group, weighted-average prediction, the abs denominator.
- ☐ Item-based CF and how it differs from user-based (the duality and why it breaks).
- ☐ The `recommend` function and its two-step stable sort.
- ☐ Evaluation: online vs. offline; MSE/RMSE/MAE/MAPE; DCG and NDCG.
- ☐ Coverage, confidence/trust, novelty, serendipity, diversity, robustness, scalability.

Chapter 9

Recommender Systems: The Utility Matrix and Approaches

Learning objectives

After this chapter you should be able to:

- define the utility matrix and explain why it is almost always sparse;
- distinguish explicit and implicit ratings and convert one to the other;
- state the four desirable properties of a good recommendation (relevant, novel, serendipitous, diverse);
- contrast content-based and collaborative filtering;
- contrast memory-based (neighborhood) and model-based (latent-factor) methods;
- state the top- k recommendation task precisely.

9.1 The utility matrix

Recommender systems were popularized by Netflix, Amazon, and Facebook. They handle two kinds of entities, *users* and *items*. Each user has a *degree of preference* for each item, expressed as a numeric *rating* or *utility*. Because most users have rated only a tiny fraction of all items, most ratings are unknown; predicting those unknowns is the central task.

Definition 9.1 (Utility matrix). A *utility matrix* M is an $m \times n$ matrix with m users as rows and n items as columns. The entry r_{ui} is the rating user u gave item i , or a missing value (the notebook uses NaN) if u has not rated i .

Why the matrix is sparse

In realistic settings there are at least 10^5 users and 10^5 items, but each user rates only a handful of items (on the order of 10). The matrix is therefore *mostly empty*. This sparsity is the single most important fact about recommender systems: any algorithm must be designed to tolerate the fact that almost every entry is unknown.

9.2 Explicit and implicit ratings

Definition 9.2 (Explicit rating). A rating the user *deliberately* assigns, e.g. a star rating from 1 to 5 on a movie. These are directly observed preferences but require user effort, so they are comparatively rare.

Definition 9.3 (Implicit rating). A preference inferred from a *proxy* behavior rather than an explicit score: whether the user watched a movie, bought an item, or clicked an ad. Implicit ratings need no user effort, so they tend to be denser, but they are noisier.

Example 9.1 (Converting explicit to implicit). Take the explicit utility matrix below (used throughout Part II). “HP” denotes Harry Potter, “TW” Twilight, “SW” Star Wars:

User	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

If we only keep “did the user consume the item?” and encode *yes* as 1, an implicit-rating version sets an entry to 1 wherever a rating existed and leaves it empty otherwise. An implicit matrix therefore has *no* notion of *dislike*: an empty cell means “unknown / not liked,” not “rated low.”

Empty is not “rated zero”

In an explicit matrix, an empty cell means “no rating yet.” In an implicit matrix, a positive value means “liked/consumed” and an empty cell means “not known to be liked.” Do not read an empty implicit cell as a 0: the system cannot distinguish “did not like” from “never encountered.”

9.3 What makes a good recommendation

The business goal is to increase revenue by surfacing relevant items. Technically, a good recommendation should be:

- **Relevant** — interesting and useful to the user;
- **Novel** — something the user has *not* seen before;
- **Serendipitous** — not immediately obvious, with an element of luck/surprise;
- **Diverse** — the list should not be a set of near-duplicates.

Definition 9.4 (Top- k recommendation task). Given a utility matrix, return the top k items to recommend to each user (or, symmetrically, the top k users to target for a given item). Operationally we predict the missing entries and recommend the k missing items with the highest predicted ratings.

9.4 Content-based vs. collaborative filtering

Definition 9.5 (Content-based filtering). Items are selected by matching *item content/features* to the *user's profile*. If a user profile says “likes red round objects,” the system scans item profiles and recommends items closest to red and round.

Definition 9.6 (Collaborative filtering (CF)). Items are recommended using *the ratings of many users acting collaboratively*, without using any description of item content.

The core idea of CF

Collaborative filtering says: *if you and I agreed on the items we both rated, then I can use your opinion on an item I have not seen to guess my own opinion*. It exploits *agreement between users* rather than *content of items*. This is why it works for items whose “content” is hard to describe (e.g. jokes, as in the Jester dataset).

9.5 Memory-based vs. model-based CF

- **Neighborhood-based** (also called *memory-based*): recommendations are computed directly from the stored ratings using a notion of similarity between rows (users) or columns (items). This notebook covers this family.
- **Model-based** (e.g. *latent factor models*): a compact model (e.g. a low-rank factorization) is *learned* from the data and then used to predict. The next notebook (Part III) covers this family.

Because preferences do not change rapidly, recommendations are usually *precomputed* and missing entries are estimated infrequently (batch, not real-time).

Chapter 10

Similarity Measures

Learning objectives

After this chapter you should be able to:

- compute raw cosine similarity between two rating vectors over common items by hand;
- explain why raw cosine is unsatisfactory and motivates mean-centering;
- compute adjusted cosine (= Pearson correlation) by hand;
- reproduce the notebook's `df_utility` matrix exactly.

Neighborhood methods rest on one ingredient: a *similarity* between two users (for user-based CF) or two items (for item-based CF). The two users (or items) are treated as vectors of ratings, but *only the coordinates that both have rated* are comparable.

10.1 The running toy matrix

The notebook defines a 4×7 utility matrix (rows = users A, B, C, D; columns = items HP1..SW3):

User	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Source discrepancy — row D of the intro table

The *introductory* markdown table (notebook cell 2) shows user D as rating HP1=3 and SW1=3. The *executable* definition (`df_utility`, cell 7) instead gives D the ratings HP2=3 and SW3=3. These are inconsistent. Throughout this reviewer (and in any exam) the *code is authoritative*: use $D = \{\text{HP2:3, SW3:3}\}$. The intro table is a sloppy illustration, not the dataset.

The exact `df_utility` construction is:

```

1 df_utility = pd.DataFrame(
2     {
3         "HP1": np.array([4, 5, np.nan, np.nan]),
4         "HP2": np.array([np.nan, 5, np.nan, 3]),
5         "HP3": np.array([np.nan, 4, np.nan, np.nan]),
6         "TW": np.array([5, np.nan, 2, np.nan]),
7         "SW1": np.array([1, np.nan, 4, np.nan]),
8         "SW2": np.array([np.nan, np.nan, 5, np.nan]),
9         "SW3": np.array([np.nan, np.nan, np.nan, 3]),
10    },
11    index=list("ABCD"),
12 )

```

Listing 10.1: Building the toy utility matrix

10.2 Raw cosine similarity

Definition 10.1 (Raw cosine similarity). Given two rating vectors a and b , let I_{ab} be the set of items both have rated. The *raw cosine* similarity is

$$\text{RawCosine}(a, b) = \frac{\sum_{i \in I_{ab}} a_i b_i}{\sqrt{\sum_{i \in I_{ab}} a_i^2} \sqrt{\sum_{i \in I_{ab}} b_i^2}},$$

with the convention that it is 0 if I_{ab} is empty or either norm is 0.

Example 10.1 (Raw cosine on the toy matrix). Using Definition 10.1:

- $\text{RawCosine}(A, B)$: common item is only HP1. Numerator $4 \cdot 5 = 20$; denominator $\sqrt{4^2} \sqrt{5^2} = 4 \cdot 5 = 20$; result $20/20 = 1$. (With a single common item, raw cosine is always 1 or -1 depending on sign.)
- $\text{RawCosine}(A, C)$: common items TW and SW1. Numerator $5 \cdot 2 + 1 \cdot 4 = 10 + 4 = 14$. Denominator $\sqrt{5^2 + 1^2} \sqrt{2^2 + 4^2} = \sqrt{26} \sqrt{20} = \sqrt{520} \approx 22.8035$. Result $14/\sqrt{520} \approx 0.6139$. The notebook asserts exactly 0.6139406135149205.
- $\text{RawCosine}(A, D)$: *no* common item, so the denominator is 0 and the function returns 0.

Raw cosine inflates similarity

Raw cosine measures the angle between the rating vectors. A user who rates everything a 5 and another who rates everything a 3 have *identical* direction, so their raw cosine is 1 — yet their *absolute* enthusiasm differs. Raw cosine also does nothing about the fact that some users are systematically generous and others harsh. Both problems are fixed by *mean-centering*.

10.3 Adjusted cosine = Pearson correlation

Definition 10.2 (Adjusted cosine similarity). Let μ_u be the mean of user u 's ratings and I_{uv} the items rated by both u and v . The *adjusted cosine* (equivalently, the *Pearson*

correlation of the two centered vectors) is

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)(r_{vi} - \mu_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \mu_v)^2}}.$$

This is exactly raw cosine applied *after subtracting each user's mean*, which is why it is called “adjusted.” The notebook’s `adjusted_cosine` computes precisely this over the common non-null items, returning 0 when the denominator is 0.

Example 10.2 (Adjusted cosine on the toy matrix). First compute each user’s mean over their rated items:

$$\mu_A = \frac{4+5+1}{3} = \frac{10}{3} \approx 3.333, \quad \mu_B = \frac{5+5+4}{3} = \frac{14}{3} \approx 4.667, \quad \mu_C = \frac{2+4+5}{3} = \frac{11}{3} \approx 3.667, \quad \mu_D = \frac{3+3}{2} = 3$$

Subtract means (keeping only rated entries):

$$\begin{aligned} A : & \text{HP1} = \frac{2}{3}, \text{TW} = \frac{5}{3}, \text{SW1} = -\frac{7}{3}; \\ B : & \text{HP1} = \frac{1}{3}, \text{HP2} = \frac{1}{3}, \text{HP3} = -\frac{2}{3}; \\ C : & \text{TW} = -\frac{5}{3}, \text{SW1} = \frac{1}{3}, \text{SW2} = \frac{4}{3}; \\ D : & \text{HP2} = 0, \text{SW3} = 0. \end{aligned}$$

Now:

- $\text{sim}(A, B)$: common item HP1. Numerator $\frac{2}{3} \cdot \frac{1}{3} = \frac{2}{9}$; denominator $\frac{2}{3} \cdot \frac{1}{3} = \frac{2}{9}$; result **1**.
- $\text{sim}(A, C)$: common items TW, SW1. Numerator $\frac{5}{3} \cdot (-\frac{5}{3}) + (-\frac{7}{3}) \cdot \frac{1}{3} = -\frac{25}{9} - \frac{7}{9} = -\frac{32}{9} \approx -3.556$. Denominator $\sqrt{(\frac{5}{3})^2 + (-\frac{7}{3})^2} \sqrt{(-\frac{5}{3})^2 + (\frac{1}{3})^2} = \sqrt{\frac{74}{9}} \sqrt{\frac{26}{9}} = \frac{\sqrt{1924}}{9} \approx 4.874$. Result $-\frac{32}{\sqrt{1924}} \approx -0.7295$. The negative sign reflects that A likes TW (5) while C dislikes it (2): they are anti-correlated.
- $\text{sim}(A, D)$: no common item $\Rightarrow$ **0**.

What mean-centering buys you

After centering, a positive product $(r_{ui} - \mu_u)(r_{vi} - \mu_v)$ means both users are *above* (or both *below*) their own average on item i — i.e. they agree on that item *relative to their own baselines*. Mean-centering removes the “generous vs. harsh rater” confound and makes similarity reflect *agreement*, not *magnitude*.

Similarity is symmetric, and ties are broken by id

$\text{sim}(u, v) = \text{sim}(v, u)$ (both the numerator and denominator are symmetric). The notebook, however, computes every ordered pair once and stores it, and later breaks ties in the neighbor list with the smaller id first. In a hand trace, always report the smaller id first when two similarities are equal.

Chapter 11

User-based Collaborative Filtering

Learning objectives

After this chapter you should be able to:

- state the user-based CF algorithm;
- write the weighted-average prediction formula and explain each term;
- explain the $\text{abs}(\text{sim})$ denominator and the discrepancy with the markdown;
- execute a full user-based completion by hand on a small matrix;
- reproduce the `recommend` two-step sort.

11.1 The algorithm

The intuition: to guess how user u would rate an item i they have not seen, find the k users *most similar* to u (the *peer group*), and take a similarity-weighted average of *those* users' ratings of i .

- Algorithm 11.1** (User-based CF — `user_complete`). 1. Compute each user's mean rating $\mu_u = \text{mean}(\text{row } u)$ over rated items.
2. **Mean-center:** replace r_{ui} by $r_{ui} - \mu_u$ (this is `df.sub(user_means, axis=0)`).
 3. For every ordered pair of distinct users u, v , compute $\text{sim}(u, v)$ (adjusted cosine over common items).
 4. For each user u , form the k *nearest neighbors* by sorting all other users v by the key $(-\text{sim}(u, v), v)$ and taking the first k .
 5. For each *missing* rating (u, i) : sum over the neighbors v that *have* rated i ,

$$\begin{aligned}\text{num} &= \sum_v \text{sim}(u, v) (r_{vi} - \mu_v), \\ \text{den} &= \sum_v |\text{sim}(u, v)|,\end{aligned}$$

and predict $\hat{r}_{ui} = \mu_u + \frac{\text{num}}{\text{den}}$. If $\text{den} = 0$ (no neighbor rated i), leave the cell `NaN`.

User-based CF — time

Let n be the number of users and m the number of items. The dominant cost is computing similarity for every ordered pair of users, each over the items both have rated (at most m): $O(n^2m)$. Forming each user's k nearest neighbors by sorting the other $n-1$ users costs $O(n^2 \log n)$, and filling the up to nm missing cells costs $O(nmk)$. Total

$$O(n^2m + n^2 \log n + nmk),$$

which is $O(n^2m)$ whenever m is the larger dimension — the usual situation for a tall rating matrix. This quadratic-in- n similarity pass is why user-based CF does not scale to huge user bases and is often replaced by item-based CF or latent-factor methods.

11.2 The prediction formula

User-based prediction

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v) (r_{vi} - \mu_v)}{\sum_{v \in N_i^k(u)} |\text{sim}(u, v)|},$$

where μ_u is user u 's mean, and $N_i^k(u)$ is the set of u 's k nearest neighbors (by adjusted cosine) that have rated item i .

Source discrepancy — the denominator is $\sum |\text{sim}|$

The markdown formula (cell 14) writes the denominator as $\sum_v \text{sim}(u, v)$ *without* absolute values, but the executable code (cell 16) uses `denominator += abs(sim)`. These differ whenever some similarity is negative. The *code is authoritative*: use the absolute value. The reason is robustness — a plain $\sum \text{sim}$ denominator can be near zero (or negative) if the peer group contains opposing tastes, which would blow up the prediction; summing $|\text{sim}|$ keeps the normalization positive.

Why “add the deviation, not the raw rating”

The numerator sums *deviations* ($r_{vi} - \mu_v$) rather than raw ratings. This predicts the *offset from u 's own mean* that the peer group implies, then adds it back to μ_u . Without this, a user who is on average harsher would inherit the generous neighbors' high raw numbers — centering transfers only the *agreement*, scaled to u 's own baseline.

11.3 Fully worked manual example

Using the toy matrix, run user-based CF with $k = 2$ for user A. From Example 15.6:

$$\text{sim}(A, B) = 1, \quad \text{sim}(A, C) \approx -0.7295, \quad \text{sim}(A, D) = 0.$$

Sort A's candidate neighbors by $(-\text{sim}, \text{id})$: B ($-\text{sim} = -1$), D ($-0 = 0$), C (0.7295), so the ordered list is $[B, D, C]$ and the top $k = 2$ are $N = \{B, D\}$.

User A's missing items are HP2, HP3, SW2, SW3. Recall the centered values (B: HP2= $\frac{1}{3}$, HP3= $-\frac{2}{3}$; D: HP2= 0, SW3= 0):

- **HP2**: B rated 5, D rated 3. $\text{num} = 1 \cdot \frac{1}{3} + 0 \cdot 0 = \frac{1}{3}$; $\text{den} = |1| + |0| = 1$; $\hat{r} = \frac{10}{3} + \frac{1}{3} = \frac{11}{3} \approx 3.667$.
- **HP3**: only B rated 4. $\text{num} = 1 \cdot (-\frac{2}{3}) = -\frac{2}{3}$; $\text{den} = 1$; $\hat{r} = \frac{10}{3} - \frac{2}{3} = \frac{8}{3} \approx 2.667$.
- **SW2**: neither B nor D rated it $\Rightarrow \text{den} = 0 \Rightarrow \text{NaN}$.
- **SW3**: only D rated 3, but $\text{sim}(A, D) = 0$ and $\text{centered}(D, \text{SW3}) = 0$, so $\text{num} = 0$, $\text{den} = 0 \Rightarrow \text{NaN}$.

So A's completed row is $[4, \frac{11}{3}, \frac{8}{3}, 5, 1, \text{NaN}, \text{NaN}]$.

Keep the fixed neighbor set, then skip unrated

Notice the peer group N is chosen *once* per user (“Find the k most similar users **ONCE**” in the code), and then, for each missing item, we *skip* neighbors that did not rate it rather than re-selecting neighbors. Reproduce this in a hand trace: fix N first, then filter per item.

11.4 The recommend function

Algorithm 11.2 (Top- n recommendation — `recommend`). Given user u , the original matrix, the completed matrix, and count n :

1. Identify the items u did *not* rate (`unrated = isna`).
2. Take their predicted ratings from the completed matrix and drop any NaN.
3. Sort *twice*: first by item id ascending (`sort_index`), then by predicted rating descending with a *stable* sort (`sort_values(ascending=False, kind="stable")`).
4. Return the first n item ids.

Top- n recommendation — time

For a single user: identifying unrated items is $O(m)$; the two sorts each cost $O(m \log m)$; extracting the first n is $O(n)$. The total is $O(m \log m)$ per user, negligible next to the $O(n^2 m)$ similarity pass that produced the completed matrix. The recommendation step is therefore cheap — the expensive work is always the neighbor/similarity computation upstream.

Why the two-pass sort

Stable sort means equal keys keep their prior order. Sorting by id first, then stably by rating, guarantees ties in rating are broken by *smallest id first* — exactly the tie rule requested in the problem. If you sorted only by rating, equal ratings would appear in arbitrary (row) order.

11.5 The Jester dataset and notebook assertions

The notebook then scales up to **Jester**, a joke-rating dataset (`jester-data-2.xls`), loaded with `pd.read_excel(..., header=None, nrows=100).iloc[:,1:]`. The raw

file uses 99 as the missing-value sentinel, so the notebook calls `df_jester.replace(99, np.nan)`. Ratings range from -10 to 10 ; the resulting matrix is 100×100 .

The notebook asserts, for `user_complete(df_jester, 5)`:

- `df_user_complete.shape == (100, 100);`
- `iloc[0,:5] == [-3.446504, 8.11, -1.135967, -2.830721, -2.28];`
- `iloc[13,:5] == [NaN, 5.8720833333333315, NaN, NaN, -0.49].`

And `recommend(5, df_jester, df_user_complete, 10)` returns `[77, 45, 80, 11, 98, 52, 25, 85, 10, 39]`.

Predictions can leave the rating range

In the toy example B's TW prediction is $\frac{19}{3} \approx 6.33$, which exceeds the 1–5 scale. Neighborhood methods have no built-in bound, so out-of-range predictions are normal; a production system might clip them to the scale. This is supplementary context — the notebook does not clip.

Chapter 12

Item-based Collaborative Filtering

Learning objectives

After this chapter you should be able to:

- state the item-based CF algorithm and how it transposes user-based CF;
- explain that the notebook still mean-centers by *user* (row), not by item;
- execute an item-based prediction by hand;
- explain the two reasons user/item duality breaks in practice.

12.1 The algorithm

Item-based CF mirrors user-based CF, but similarity is computed between *columns* (items) rather than *rows* (users): two items are similar if they are rated similarly *by the same users*.

Algorithm 12.1 (Item-based CF — `item_complete`). 1. Compute each *user*’s mean μ_u (row mean) and center the matrix by *rows*: $r_{ui} \leftarrow r_{ui} - \mu_u$. (Note: the centering is still by user, not by item.)

2. For every pair of distinct items i, j , compute $\text{sim}(i, j)$ (adjusted cosine over the *users* who rated both).
3. For each item i , form its k nearest neighbors by sorting items j by $(-\text{sim}(i, j), j)$ and taking the first k .
4. For each missing (u, i) : over neighbors j that user u has rated, accumulate $\text{num} = \sum_j \text{sim}(i, j) (r_{uj} - \mu_u)$ and $\text{den} = \sum_j |\text{sim}(i, j)|$, then $\hat{r}_{ui} = \mu_u + \text{num}/\text{den}$; leave **NaN** if $\text{den} = 0$.

Item-based CF — time

Mirroring user-based CF with the roles of users and items swapped: similarity is computed for every pair of items over the users who rated both, $O(m^2n)$, and each item’s k nearest neighbors cost $O(m^2 \log m)$ to select. Filling missing cells costs $O(nmk)$. Total

$$O(m^2n + m^2 \log n + nmk),$$

i.e. $O(m^2n)$ when n is the larger dimension. This is the reason item-based CF is preferred over user-based CF when there are *few items but many users*: the quadratic pass is over m , not n , so it stays tractable as the user base grows.

Centering direction is still by user

A natural-looking mistake is to center each *item's* column (subtract the item mean) when doing item-based CF. The notebook does *not*: it computes `user_means = df_utility.mean(axis=1)` (row means) and subtracts those, in both user-based and item-based versions. This is the “adjusted cosine” convention — the centering always removes the *rater's* baseline. Follow the code.

12.2 Fully worked manual example

Predict A's missing rating for HP2 with $k = 2$. First compute the item-item similarities involving HP2, using the user-centered matrix of Example 15.6:

- $\text{sim}(\text{HP2}, \text{HP1})$: common user B; $\frac{1}{3} \cdot \frac{1}{3}$ over $(\frac{1}{3})(\frac{1}{3}) = 1$;
- $\text{sim}(\text{HP2}, \text{HP3})$: common user B; $\frac{1}{3} \cdot (-\frac{2}{3})$ over $(\frac{1}{3})(\frac{2}{3}) = -1$;
- $\text{sim}(\text{HP2}, \text{TW}) = \text{sim}(\text{HP2}, \text{SW1}) = \text{sim}(\text{HP2}, \text{SW2}) = 0$ (no common user);
- $\text{sim}(\text{HP2}, \text{SW3})$: common user D, both centered to 0 $\Rightarrow 0$.

Sorting HP2's neighbors by $(-\text{sim}, \text{id})$ gives $[\text{HP1}(-1), \text{TW}(0), \text{SW1}(0), \text{SW2}(0), \text{SW3}(0), \text{HP3}(1)]$, so the top $k = 2$ are $\{\text{HP1}, \text{TW}\}$. For user A, who rated $\text{HP1}=4$ and $\text{TW}=5$:

$$\text{num} = \underbrace{1 \cdot \frac{2}{3}}_{\text{HP1}} + \underbrace{0 \cdot \frac{5}{3}}_{\text{TW}} = \frac{2}{3}, \quad \text{den} = |1| + |0| = 1, \quad \hat{r}_{A, \text{HP2}} = \frac{10}{3} + \frac{2}{3} = 4.$$

Filter neighbors by “did this user rate it?”

The neighbor set is fixed per *item*; then for a given user we keep only the neighbors that user rated. In the example, TW is a top-2 neighbor of HP2, but A *has* rated TW, so it is (harmlessly) included with similarity 0. A neighbor with similarity 0 contributes nothing but still adds $|0|$ to the denominator, so watch for it in a hand trace.

12.3 User-based vs. item-based: the broken duality

The two algorithms are nearly identical — user-based ranks rows, item-based ranks columns. In practice the symmetry breaks for two reasons (notebook cell 26):

1. **Extra step.** Once you have identified the most similar *users*, you can recommend immediately; with only the most similar *items* you need an extra aggregation step over a user's items.
2. **Genre structure.** Items cluster into genres; a user may like several genres. It is easier to find similar items (they share a genre) than similar users (who may overlap in one genre but differ elsewhere).

When to use which

Item-based similarity tends to be more *stable* and is often precomputed for large catalogs because item–item similarity changes slowly. User-based similarity is more volatile (a new rating changes the user’s vector) but needs no item profile. Both are valid; the choice is usually driven by whether the number of users or the number of items is larger, and by how fast preferences change.

The notebook asserts for `item_complete(df_jester, 5):` `iloc[0,:5] == [7.09, 8.11, -7.28, NaN, -2.28]` and `iloc[13,:5] == [6.650871400844572, NaN, 0.924763052000898, NaN, -0.49]`.

Chapter 13

Evaluating Recommender Systems

Learning objectives

After this chapter you should be able to:

- contrast online and offline evaluation;
- compute MSE, RMSE, MAE, and MAPE from a list of actual vs. predicted ratings;
- compute DCG and NDCG by hand;
- define and give examples of coverage, confidence, trust, novelty, serendipity, diversity, robustness, and scalability.

13.1 Online vs. offline evaluation

- **Online**: users interact with a running system and their feedback is used for evaluation (recruited users, or real users in an A/B test). Expensive, hard to compare across systems, but realistic.
- **Offline**: use already-collected data (e.g. a held-out subset of ratings). Simple and cheap (the one used in research), but risks being *stale* and biased by what the old system chose to show.

13.2 Desirable properties

Besides accuracy, a recommender system is evaluated on:

- **Coverage** — how many users can receive n recommendations (users need a minimum number of ratings first).
- **Confidence** — how narrow the confidence interval around a predicted rating is.
- **Trust** — how much a user believes the predictions.
- **Novelty** — likelihood of surfacing items the user is not yet aware of.
- **Serendipity** — level of (pleasant) surprise in successful recommendations.

- **Diversity** — the recommended items should not be near-duplicates.
- **Robustness / Stability** — resilience to attacks and to drift in user behavior.
- **Scalability** — efficiency on large data.

The course focuses on measuring *accuracy* (of ratings and of rankings).

13.3 Accuracy of ratings: MSE, RMSE, MAE, MAPE

Predicting ratings is a *regression* problem, so regression metrics apply. Let r_i be the true rating and $\hat{r}_i$ the prediction, over n test ratings.

Regression accuracy metrics

$$\text{MAE} = \frac{1}{n} \sum_i |\hat{r}_i - r_i|, \quad \text{MSE} = \frac{1}{n} \sum_i (\hat{r}_i - r_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}},$$

$$\text{MAPE} = \frac{100}{n} \sum_i \left| \frac{\hat{r}_i - r_i}{r_i} \right| \quad (\text{as a percentage}).$$

Example 13.1 (Computing the metrics by hand). Actual ratings $r = [5, 3, 4, 2]$, predictions $\hat{r} = [4, 3, 5, 2]$. Errors $\hat{r} - r = [-1, 0, 1, 0]$.

$$\text{MAE} = \frac{1+0+1+0}{4} = 0.5, \quad \text{MSE} = \frac{1+0+1+0}{4} = 0.5, \quad \text{RMSE} = \sqrt{0.5} \approx 0.707.$$

(The notebook mentions these four metrics but leaves their formulas to the reader; the definitions above are standard.)

RMSE penalizes large errors

RMSE squares the error before averaging, so it penalizes large errors more than MAE does. If two systems have the same MAE but one has occasional large misses, that one has a larger RMSE. MAPE normalizes by the true value, so it is misleading when some true ratings are 0 (division by zero).

13.4 Accuracy of ranking: DCG and NDCG

Reconstructing every rating is not the real goal — what matters is whether the *small set of items we actually recommend* is useful, with earlier items more important. *Discounted Cumulative Gain* (DCG) and its normalized version (NDCG) capture exactly this.

Definition 13.1 (DCG and NDCG). Let L be the number of recommended items, $v_j \in \{1, \dots, L\}$ the *rank* of item j in the recommendation, r_{uj} the *actual* rating of user u on item j , and I_u the items rated by u . Then

$$\text{DCG} = \frac{1}{m} \sum_{u=1}^m \sum_{\substack{j \in I_u \\ v_j \leq L}} \frac{2^{r_{uj}} - 1}{\log_2(v_j + 1)}, \quad \text{NDCG} = \frac{\text{DCG}}{\text{IDCG}},$$

where IDCG is the DCG of the *ideal* ranking (the user's own top- L rated items, in descending order of their true rating).

Reading the gain and the discount

The gain $2^r - 1$ grows *exponentially* in the rating, so a top rating contributes far more than a mediocre one. The discount $1/\log_2(v_j + 1)$ shrinks with rank, so getting the right item *first* matters most. This is the ranking analogue of “precision at the top.”

Example 13.2 (DCG and NDCG by hand). One user ($m = 1$) has true ratings $c = 5$, $a = 3$, $d = 2$, $b = 1$; the system recommends, in order, $[c, a, e]$ ($L = 3$, item e unrated).

- rank 1, item $c = 5$: $\frac{2^5 - 1}{\log_2 2} = \frac{31}{1} = 31$;
- rank 2, item $a = 3$: $\frac{2^3 - 1}{\log_2 3} = \frac{7}{1.5850} \approx 4.416$;
- rank 3, item e : unrated $\Rightarrow$ skipped.

DCG = $31 + 4.416 \approx \mathbf{35.416}$.

For IDCG, the ideal top-3 is the user’s own ratings sorted descending: $[c = 5, a = 3, d = 2]$:

$$\frac{31}{\log_2 2} + \frac{7}{\log_2 3} + \frac{2^2 - 1}{\log_2 4} = 31 + 4.416 + \frac{3}{2} = 36.916.$$

So NDCG = $\frac{35.416}{36.916} \approx \mathbf{0.9594}$. The recommendation is excellent — it got the single best item first; it only “missed” by placing an unrated item e in slot 3 instead of the rated item d .

The gain $2^r - 1$ goes negative for negative ratings

The standard DCG gain is designed for *non-negative* relevance. The Jester ratings run from -10 to 10 , so for any $r < 0$ the gain $2^r - 1$ is negative (e.g. $2^{-2} - 1 = -0.75$). A disliked joke in a recommendation therefore *subtracts* from DCG. This is a quirk of applying DCG to signed ratings; the notebook does not adjust for it, so reproduce it as-is and be ready to point the quirk out if asked.

13.5 Notebook assertions for DCG/NDCG

The notebook’s `dcg(df, recos)` divides the total gain by m (the number of users), and `ndcg` returns `total_dcg/total_idcg` (a *sum* ratio over users, not a per-user mean). Its assertions:

- `dcg(user 5) = 23.023067518292404`;
- `dcg(users 5, 19) = 101.86738320819285`;
- `ndcg(user 5) = 0.23169495849443336`;
- `ndcg(users 5, 19) = 0.10609066993151361`.

DCG averages per user; NDCG is a global ratio

dcg returns $\frac{1}{m} \sum_u \text{DCG}_u$ (mean over users), but **ndcg** returns $\frac{\sum_u \text{DCG}_u}{\sum_u \text{IDCG}_u}$ (the ratio of the sums). Do not average the per-user NDCGs. This distinction is a frequent source of arithmetic errors.

Chapter 14

Summary: Formulas, Algorithms, and Exam Strategy

14.1 Formula sheet (Part II)

Raw cosine over common items

$$\text{RawCosine}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2} \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}, \text{ with value 0 if the denominator is 0.}$$

Adjusted cosine = Pearson

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)(r_{vi} - \mu_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \mu_v)^2}}.$$

Weighted-average prediction (user- and item-based)

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v)(r_{vi} - \mu_v)}{\sum_{v \in N_i^k(u)} |\text{sim}(u, v)|}. \text{ (For item-based, swap roles: } \text{sim}(i, j) \text{ over common users, deviations } r_{uj} - \mu_u.)$$

Accuracy and ranking metrics

$$\text{MAE} = \frac{1}{n} \sum |\hat{r} - r|, \quad \text{MSE} = \frac{1}{n} \sum (\hat{r} - r)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAPE} = \frac{100}{n} \sum \left| \frac{\hat{r} - r}{r} \right|.$$

$$\text{DCG} = \frac{1}{m} \sum_u \sum_{\substack{j \in I_u \\ v_j \leq L}} \frac{2^{r_{uj}} - 1}{\log_2(v_j + 1)}, \quad \text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}.$$

14.2 Algorithm summaries

- **User-based CF** — mean-center rows; pairwise adjusted-cosine similarity over common items; pick k nearest neighbors per user (fixed, then filter per item); weighted-

average of deviations with $|\text{sim}|$ denominator.

- **Item-based CF** — same, but similarity over common *users* between *columns*; still mean-center by user; pick k nearest items; aggregate over the user's rated neighbors.
- **recommend** — unrated items $\rightarrow$ drop unpredicted $\rightarrow$ sort by id then stably by rating desc $\rightarrow$ top- n .
- **Evaluation** — regression: MSE/RMSE/MAE/MAPE; ranking: DCG/NDCG.

14.3 Decision guide

- Asked to *compute similarity*? Check whether it is raw (no centering) or adjusted (centered). For one common item, similarity is ± 1 (or 0 if both centered to 0).
- Asked to *run user-based CF*? Fix the peer group once, then for each missing cell skip unrated neighbors.
- Asked to *run item-based CF*? Center by *user*, then similarity over common *users*.
- Asked to *produce top- n* ? Remember the id-first, stable-rating-second sort.
- Asked to *evaluate ranking*? Use DCG/NDCG, not RMSE; compute IDCG from the user's own top- L ratings.

14.4 Common exam traps and time savers

Most common mistakes

- Computing similarity over *all* items instead of only the *common* (both-rated) items.
- Forgetting to mean-center before adjusted cosine, or centering by *item* in item-based CF.
- Using a *signed* denominator $\sum \text{sim}$ instead of $\sum |\text{sim}|$.
- Averaging per-user NDCG instead of taking the global ratio.
- Treating an empty implicit cell as “rating 0.”
- Forgetting to *drop* unpredicted (NaN) cells in **recommend**.

Time savers

- For a single common item, adjusted cosine is ± 1 (sign = sign of product) — no arithmetic needed.
- $\log_2 2 = 1$, $\log_2 4 = 2$; only $\log_2 3 \approx 1.585$ needs remembering for small DCG examples.
- $2^r - 1$: $2^5 - 1 = 31$, $2^4 - 1 = 15$, $2^3 - 1 = 7$, $2^2 - 1 = 3$, $2^1 - 1 = 1$, $2^0 - 1 = 0$.
- Sanity check a prediction: it should be a weighted average of neighbor deviations around μ_u ; if all deviations have the same sign, the prediction moves μ_u in that direction.

Prerequisite reminders

- Mean of a vector with missing values = mean of its non-missing entries.
- $\sqrt{a^2 + b^2}$ is the Euclidean norm; cosine is dot product over norms.
- A stable sort preserves the prior order among equal keys.

Chapter 15

Practice Problems — Part II

Solutions and point-based rubrics are in Part IV (the answer key). Attempt every problem before consulting it. All manual problems use the 4×7 `df_utility` matrix of this part unless another matrix is given.

15.1 Category A — Conceptual

Exercise 15.1 (Utility matrix and rating types). Category A — Conceptual Easy, 3 min Define a utility matrix. Distinguish explicit and implicit ratings, and explain why a utility matrix is almost always sparse.

Exercise 15.2 (Content-based vs. collaborative). Category A — Conceptual Easy, 3 min Contrast content-based and collaborative filtering. Give one advantage of collaborative filtering for items (such as jokes) whose content is hard to describe.

Exercise 15.3 (Neighborhood vs. model-based; user vs. item). Category A — Conceptual Moderate, 4 min Distinguish memory-based (neighborhood) from model-based CF, and user-based from item-based CF. State the two reasons the user/item duality usually breaks in practice.

Exercise 15.4 (Why mean-center?). Category A — Conceptual Moderate, 4 min Explain why we mean-center before computing adjusted cosine, and why raw cosine is considered a poor similarity measure on its own. Give a concrete pair of rating vectors that raw cosine would rate as perfectly similar but that should not be.

15.2 Category B — Basic computation

Exercise 15.5 (Raw cosine). Category B — Basic computation Easy, 4 min Compute $\text{RawCosine}(B, D)$ and $\text{RawCosine}(B, C)$ on `df_utility`. Show the common-item set and the arithmetic for each.

Exercise 15.6 (Adjusted cosine). Category B — Basic computation Moderate, 6 min On `df_utility`, compute the adjusted cosine (Pearson) similarities $\text{sim}(A, C)$ and $\text{sim}(B, D)$. Give each as an exact fraction and an approximate decimal.

Exercise 15.7 (Regression metrics). Category B — Basic computation Easy, 4 min True ratings $r = [1, 4, 2, 5]$, predictions $\hat{r} = [2, 4, 1, 4]$. Compute MAE, MSE, and RMSE. Which prediction contributes most to the MSE?

Exercise 15.8 (DCG of one user). Category B — Basic computation Moderate, 5 min One user has true ratings $\{x : 5, y : 3, z : 2, w : 4\}$ (items w, x, y, z). The system recommends $[x, y, w]$ ($L = 3$). Compute the DCG (with $m = 1$) and the IDCG, and hence the NDCG. Report intermediate terms.

15.3 Category C — Manual algorithm execution

Exercise 15.9 (User-based CF by hand). Category C — Manual algorithm execution Moderate, 10 min On `df_utility`, run user-based CF with $k = 2$ to complete user B's row (missing items TW, SW1, SW2, SW3). List B's neighbor ordering, and show numerator/denominator for each missing cell.

Exercise 15.10 (Item-based CF by hand). Category C — Manual algorithm execution Moderate, 8 min On `df_utility`, run item-based CF with $k = 2$ to predict A's rating of HP2. List HP2's neighbor ordering and the arithmetic.

Exercise 15.11 (NDCG by hand). Category C — Manual algorithm execution Moderate, 8 min Two users:

- User 1 ratings: $\{a : 5, b : 2, c : 4\}$; recommendation $[c, a]$.
- User 2 ratings: $\{a : 1, b : 5, c : 2\}$; recommendation $[b, a]$.

With $L = 2$ for both, compute DCG over both users and NDCG over both users, using the notebook's exact definitions (mean for DCG, global ratio for NDCG).

15.4 Category D — Multi-step / integrated

Exercise 15.12 (Full pipeline and recommendation). Category D — Multi-step / integrated Difficult, 15 min On `df_utility`, run user-based CF with $k = 2$ to complete *all four* users, then use `recommend` to return the top-2 recommendations for user A and for user B. Report the full completed matrix and justify each returned id.

15.5 Category E — Challenging

Exercise 15.13 (Coverage and cold start). Category E — Challenging Moderate, 6 min Define user-space coverage. In `df_utility`, which user(s) receive *no* predictions under user-based CF with $k = 2$, and why? Relate your answer to the concept of coverage and to the cold-start problem.

Exercise 15.14 (The abs denominator). Category E — Challenging Difficult, 10 min The code's prediction denominator is $\sum |\text{sim}|$, but the markdown formula omits the absolute value. (a) Explain what could go wrong with the unsigned denominator $\sum \text{sim}$. (b) Suppose a user's peer group contains only one neighbor with similarity -0.5 ; compute the prediction both ways and discuss which is more sensible.

Part III

Latent-factor Collaborative Filtering

Part III — Topic Checklist

- ☐ The UV (latent-factor) decomposition $M \approx F_{\text{user}} F_{\text{item}}^\top$ and what d (latent factors) means.
- ☐ The Frobenius-norm objective $J = \frac{1}{2} \sum_{(i,j): r_{ij} \neq \emptyset} (r_{ij} - \sum_s u_{is} v_{js})^2$.
- ☐ The gradients $\partial J / \partial u_{iq}$, $\partial J / \partial v_{jq}$ and the error e_{ij} .
- ☐ Batch gradient descent vs. stochastic gradient descent (SGD) update rules.
- ☐ Alternating Least Squares (ALS): alternating closed-form row solves, and the all-ones first-iteration example.
- ☐ Coordinate Gradient Descent (CGD): optimizing one element of U or V at a time, the u_{ij}/v_{ij} closed forms.
- ☐ The storage advantage of storing $(n+k)d$ factor entries instead of nk .
- ☐ The **Surprise** library: `KNNWithMeans`, `Reader`, long-format melt, trainset/testset/anti-testset.

Chapter 16

The Latent-Factor Model: UV Decomposition

Learning objectives

After this chapter you should be able to:

- state the UV decomposition $M \approx F_{\text{user}} F_{\text{item}}^\top$ and the shapes involved;
- explain what a latent factor is and why the model “completes” the matrix;
- write the Frobenius-norm objective restricted to observed entries;
- derive the partial derivatives with respect to a single u_{iq} and v_{jq} .

16.1 From matrix completion to matrix factorization

In Part II we *completed* the utility matrix one missing cell at a time using neighborhood similarity. Latent-factor methods take a different, global view: they assume the entire matrix is (approximately) the product of two low-rank matrices, and they *learn* those factors to fit the observed ratings.

Definition 16.1 (UV decomposition). Let the utility matrix M be $n \times k$ (n users, k items). A *latent-factor* (or *UV*) decomposition writes

$$M \approx F_{\text{user}} F_{\text{item}}^\top,$$

where F_{user} is $n \times d$ and F_{item} is $k \times d$. The integer d is the number of *latent components* (latent factors), and is typically $d \ll \min(n, k)$.

What a latent factor is

Each user is summarized by a length- d vector u_i of “coordinates” along d hidden dimensions; each item by a length- d vector v_j . The predicted rating is the dot product $u_i \cdot v_j = \sum_s u_{is} v_{js}$. The dimensions are *latent* (hidden, not directly observed): they might roughly correspond to “prefers action,” “dry humor,” “polish quality,” etc., but the algorithm discovers them automatically and does not name them. Multiplying the two factor matrices reconstructs (completes) every entry of M , observed and unobserved alike.

16.2 The objective function

Definition 16.2 (Frobenius-norm objective). We find $U = F_{\text{user}}$ and $V = F_{\text{item}}$ to minimize

$$J = \frac{1}{2} \|M - UV^\top\|_F^2 = \frac{1}{2} \sum_{(i,j): r_{ij} \neq \emptyset} \left(r_{ij} - \sum_{s=1}^d u_{is} v_{js} \right)^2,$$

where the sum runs *only over observed (rated) entries* $r_{ij} \neq \emptyset$, and $\|\cdot\|_F$ is the Frobenius norm.

Only rated entries count

The objective sums squares only where r_{ij} is known. This is the crucial detail: we never try to match the (unknown) empty cells directly; instead, by fitting the observed cells, the factor model *induces* predictions on the empty cells through the same $u_i \cdot v_j$ formula.

16.3 Gradients

Let $e_{ij} = r_{ij} - \sum_s u_{is} v_{js}$ be the *residual (error)* on the known entry (i, j) . Holding the other factor fixed, the partial derivatives are

Gradients of J

$$\begin{aligned} \frac{\partial J}{\partial u_{iq}} &= \sum_{j: r_{ij} \neq \emptyset} \left(r_{ij} - \sum_s u_{is} v_{js} \right) (-v_{jq}) = \sum_{j: r_{ij} \neq \emptyset} e_{ij} (-v_{jq}), \\ \frac{\partial J}{\partial v_{jq}} &= \sum_{i: r_{ij} \neq \emptyset} \left(r_{ij} - \sum_s u_{is} v_{js} \right) (-u_{iq}) = \sum_{i: r_{ij} \neq \emptyset} e_{ij} (-u_{iq}). \end{aligned}$$

Sketch of the u_{iq} derivative. Only the term $(r_{ij} - \sum_s u_{is} v_{js})^2$ for fixed i involves u_{iq} , and $\frac{\partial}{\partial u_{iq}}(r_{ij} - \sum_s u_{is} v_{js}) = -v_{jq}$. Applying the chain rule to $\frac{1}{2}(\cdot)^2$ gives the factor $(r_{ij} - \sum_s u_{is} v_{js})(-v_{jq})$; summing over j with $r_{ij} \neq \emptyset$ yields the result. The v_{jq} case is symmetric. $\square$

Chapter 17

Gradient Descent: Batch and Stochastic

Learning objectives

After this chapter you should be able to:

- write the batch gradient-descent update (including the matrix form $U \leftarrow U + \alpha EV$);
- write the SGD update for a single observed pair (i, j) ;
- contrast batch GD and SGD (cost per step, stability, speed);
- execute a small SGD trace by hand.

17.1 Batch gradient descent

Gradient descent moves the parameters *against* the gradient. Since $\frac{\partial J}{\partial u_{iq}} = -\sum_j e_{ij}v_{jq}$, the update for the *full* gradient is

$$u_{iq} \leftarrow u_{iq} + \alpha \sum_{j: r_{ij} \neq \emptyset} e_{ij}v_{jq}, \quad v_{jq} \leftarrow v_{jq} + \alpha \sum_{i: r_{ij} \neq \emptyset} e_{ij}u_{iq},$$

where $\alpha > 0$ is the learning rate.

Batch GD in matrix form

Let E be the $n \times k$ error matrix with $E_{ij} = e_{ij}$ on observed entries (and 0 otherwise). Then one batch step is

$$U \leftarrow U + \alpha EV, \quad V \leftarrow V + \alpha E^\top U.$$

Batch gradient descent — time

Let $|\Omega|$ be the number of observed (rated) entries. One iteration must compute every residual e_{ij} ($O(|\Omega|d)$, one dot product each) and then accumulate the two gradient sums; in matrix form these are the sparse products EV and $E^\top U$, each $O(|\Omega|d)$ because E has exactly $|\Omega|$ non-zeros. Thus one batch step is

$$O(|\Omega|d) \text{ per iteration,}$$

with memory $O(nd + kd)$ for the two factor matrices. The catch is convergence: a full sweep uses *all* data before every update, so many sweeps are usually needed.

Why “ EV ” and “ $E^\top V$ ”

$(EV)_{iq} = \sum_j E_{ij} V_{jq} = \sum_j e_{ij} v_{jq}$ reproduces exactly the gradient-sum above. Transposing to $E^\top U$ swaps the roles: $(E^\top U)_{jq} = \sum_i e_{ij} u_{iq}$, the gradient for v_{jq} .

17.2 Stochastic gradient descent (SGD)

Batch GD must touch *all* observed ratings before each update, which is expensive and slow to converge. SGD instead updates using a *single* observed pair (i, j) at a time:

SGD update for one observed pair

For a single observed rating (i, j) with error $e_{ij} = r_{ij} - u_i \cdot v_j$:

$$u_i \leftarrow u_i + \alpha e_{ij} v_j, \quad v_j \leftarrow v_j + \alpha e_{ij} u_i.$$

Stochastic gradient descent — time

One SGD update uses a single observed pair (i, j) : it computes the dot product $u_i \cdot v_j$ ($O(d)$) and updates the two length- d vectors u_i and v_j ($O(d)$ each), so

$$O(d) \text{ per rating.}$$

A full *epoch* touches all $|\Omega|$ observed entries once, hence $O(|\Omega|d)$ — the same total arithmetic as one batch step, but delivered as many small, independent updates that converge faster in wall-clock time and can escape poor local minima.

Batch vs. SGD in one line

Batch GD: one update per full pass, uses the whole matrix, smoother but slower.
SGD: one update per observed rating, uses one entry, noisier but much faster and can escape poor local minima. ALS (next chapter) is the *stable* closed-form alternative that avoids choosing a learning rate at all.

17.3 Small SGD trace

Take $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$ (all observed), $d = 1$, and initial $U = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $V = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ (so $u_1 = u_2 = 1$, $v_1 = 1$, $v_2 = 2$). With $\alpha = 0.1$, process entry $(1, 1)$ then $(1, 2)$:

- $(1, 1)$: $e_{11} = 5 - u_1 v_1 = 5 - 1 = 4$. $u_1 \leftarrow 1 + 0.1 \cdot 4 \cdot 1 = 1.4$; $v_1 \leftarrow 1 + 0.1 \cdot 4 \cdot 1 = 1.4$.
- $(1, 2)$: $e_{12} = 2 - u_1 v_2 = 2 - 1.4 \cdot 2 = -0.8$. $u_1 \leftarrow 1.4 + 0.1 \cdot (-0.8) \cdot 2 = 1.24$; $v_2 \leftarrow 2 + 0.1 \cdot (-0.8) \cdot 1.4 = 1.888$.

After one pass, $U = \begin{pmatrix} 1.24 \\ 1 \end{pmatrix}$, $V = \begin{pmatrix} 1.4 \\ 1.888 \end{pmatrix}$.

Chapter 18

Alternating Least Squares (ALS)

Learning objectives

After this chapter you should be able to:

- state the ALS algorithm and why each sub-problem is a least-squares solve;
- explain the parallelization argument;
- reproduce the all-ones first-iteration numbers on the 5×5 example;
- write the `als` pseudocode and the RMSE/tolerance stopping rule.

18.1 The algorithm

ALS is more stable than SGD because each step is an *exact* least-squares solve rather than a gradient step. It alternates:

Algorithm 18.1 (ALS). Repeat until the RMSE on observed entries falls below a tolerance `tol`:

1. **Fix U , solve for V .** For each item j , keep $u_{i1}, \dots, u_{id}$ constant and minimize $\sum_{i: r_{ij} \neq \emptyset} (r_{ij} - \sum_s u_{is} v_{js})^2$ over v_j (a least-squares problem with d unknowns). This is one independent problem per item, so they are *parallelizable*.
2. **Fix V , solve for U .** For each user i , minimize $\sum_{j: r_{ij} \neq \emptyset} (r_{ij} - \sum_s u_{is} v_{js})^2$ over u_i . One independent problem per user, also *parallelizable*.
3. Compute $\text{RMSE} = \sqrt{\text{nanmean}((M - UV^\top)^2)}$; stop if $\text{RMSE} < \text{tol}$.

ALS — time

One iteration solves n user sub-problems and k item sub-problems, each a least-squares solve in d unknowns. For user i with $|\Omega_i|$ ratings, forming the normal-equation matrix $\sum_j v_j v_j^\top$ costs $O(|\Omega_i| d^2)$ and solving the resulting $d \times d$ system costs $O(d^3)$. Summing over all users and items,

$$O(|\Omega| d^2 + (n + k) d^3) \quad \text{per iteration.}$$

The d^2 term comes from the data (the ratings), the d^3 term from inverting each small system. Since d is small this is fast per iteration; the trade-off is that ALS may need

many iterations to converge, and it avoids any learning rate (see the exam tip below).

ALS row solve (normal equations)

With V fixed, the optimal row u_i for user i minimizes over u : $\frac{1}{2} \sum_{j: r_{ij} \neq \emptyset} (r_{ij} - u v_j)^2$; in closed form,

$$u_i = \left(\sum_{j: r_{ij} \neq \emptyset} v_j v_j^\top \right)^{-1} \left(\sum_{j: r_{ij} \neq \emptyset} r_{ij} v_j \right),$$

with the symmetric formula for v_j holding U fixed. (The notebook uses `scipy.linalg.lstsq`, which also handles rank-deficient cases by returning a minimum-norm solution.)

Notebook wording has swapped indices

The markdown (cell 5) says “solve for each of the d rows of V ” and “each of the n rows of U ,” and later writes “ $u_{i1} \dots u_{in}$ constant.” These are index slips: V has k rows (one per *item*) each of length d , and U has n rows (one per *user*); the constant factor runs over the d components, not n or k . The intended algorithm — and the executable `als` function — is the one stated above.

18.2 Fully worked first iteration

Example 18.1 (ALS first iteration on the 5×5 matrix). The utility matrix (same one used for CGD later) is

$$M = \begin{pmatrix} 5 & 2 & 4 & 4 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ - & - & 3 & 1 & 4 \\ 2 & 5 & 4 & 3 & 5 \\ 4 & 4 & 5 & 4 & - \end{pmatrix},$$

where “ $-$ ” denotes an empty (unrated) cell, and we use $d = 2$ latent factors. Initialize U to all ones (5×2).

Step 1 (solve V). With every $u_i = [1, 1]$, the predicted rating $u_i \cdot v_j = v_{j1} + v_{j2}$ depends only on the *sum* of item j ’s two factors. The least-squares/minimum-norm solution splits that sum evenly, giving each item j the vector $v_j = [\frac{\bar{m}_j}{2}, \frac{\bar{m}_j}{2}]$, where $\bar{m}_j$ is the mean of item j ’s observed ratings:

item j	observed ratings	$\bar{m}_j$
0	5, 3, 2, 4	$14/4 = 3.5$
1	2, 1, 5, 4	$12/4 = 3.0$
2	4, 2, 3, 4, 5	$18/5 = 3.6$
3	4, 4, 1, 3, 4	$16/5 = 3.2$
4	3, 1, 4, 5	$13/4 = 3.25$

Hence $V^\top$ (the $d \times k$ transposed form shown in the notebook) is

$$\begin{pmatrix} 1.75 & 1.5 & 1.8 & 1.6 & 1.625 \\ 1.75 & 1.5 & 1.8 & 1.6 & 1.625 \end{pmatrix},$$

i.e. $v_0 = [1.75, 1.75]$, $v_1 = [1.5, 1.5]$, $v_2 = [1.8, 1.8]$, $v_3 = [1.6, 1.6]$, $v_4 = [1.625, 1.625]$.

Step 2 (solve U). Now $v_j = [\bar{m}_j/2, \bar{m}_j/2]$, so $u_i \cdot v_j = (u_{i1} + u_{i2})\frac{\bar{m}_j}{2}$. Let $s_i = u_{i1} + u_{i2}$; the least-squares optimum for the scalar s_i is

$$s_i = \frac{\sum_{j:r_{ij} \neq \emptyset} r_{ij} \frac{\bar{m}_j}{2}}{\sum_{j:r_{ij} \neq \emptyset} \left(\frac{\bar{m}_j}{2}\right)^2},$$

and the minimum-norm split gives $u_{i1} = u_{i2} = s_i/2$. For user 0 (rated all five items, $r = [5, 2, 4, 4, 3]$):

$$\sum r_{ij} \frac{\bar{m}_j}{2} = 5(1.75) + 2(1.5) + 4(1.8) + 4(1.6) + 3(1.625) = 30.225,$$

$$\sum \left(\frac{\bar{m}_j}{2}\right)^2 = 1.75^2 + 1.5^2 + 1.8^2 + 1.6^2 + 1.625^2 = 13.753125,$$

so $s_0 = 30.225/13.753125 = 2.19768$, giving $u_0 = [1.09884117, 1.09884117]$. Carrying this out for every user yields the notebook's first-iteration result:

$$U = \begin{pmatrix} 1.09884117 & 1.09884117 \\ 0.66802999 & 0.66802999 \\ 0.79970381 & 0.79970381 \\ 1.13156101 & 1.13156101 \\ 1.27784027 & 1.27784027 \end{pmatrix}.$$

Why the factors split evenly

Because U starts at all ones, the two latent columns are initially identical, so nothing distinguishes factor 1 from factor 2. The minimum-norm solution therefore assigns equal weight to both. On later iterations the two columns diverge as the least-squares problems use a non-constant U .

18.3 The `als` function and recommendation

The executable `als` (cell 7) loops: solve all V rows via `lstsq`, solve all U rows via `lstsq`, compute RMSE with `np.nanmean`, and return (U, V) once $\text{RMSE} < \text{tol}$. Both matrices are initialized to all ones (to make the asserts deterministic).

The recommendation function (cell 9, Problem 1) builds the dense prediction matrix $P = UV^\top$, masks out the items the user already rated, sorts the remaining items by predicted rating descending (with ties in ascending id order, via the stable sort), and returns the first L . On Jester with $d = 50$, $\text{tol}=2$: `recommend_als(5, ..., 10)` returns `[51, 55, 1, 70, 30, 63, 9, 52, 43, 59]`.

ALS has no learning rate

A frequent conceptual question asks how ALS differs from gradient descent. The key answer: ALS never needs a learning rate α — each sub-step is an exact least-squares solution, so there is no step-size tuning and no divergence risk. Its cost is solving many linear systems per sweep.

Chapter 19

Coordinate Gradient Descent (CGD)

Learning objectives

After this chapter you should be able to:

- state the CGD idea (fix everything except one element, optimize it);
- reproduce the $x = 2.6$ and $y = 1.68$ worked example;
- derive the closed forms for u_{ij} and v_{ij} ;
- explain the storage advantage of the factor matrices.

19.1 The idea

ALS is parallelizable but not as efficient as SGD; SGD is efficient but less stable. CGD occupies the middle: fix *both* factor matrices except for a *single element*, and optimize that one element in closed form. Sweeping over all elements of U then V (the code does this row-by-row, left-to-right) until the fractional SSE improvement drops below `tol` gives the factorization.

19.2 Worked example: one element of U , then one of V

Example 19.1 (CGD first two coordinates). Initialize U (all ones, 5×2) and V (all ones, 5×2) on the same 5×5 matrix M of Example 18.1. We first optimize u_{00} (the upper-left element of U), call it x . Because only row 0 of the product depends on x , and $v_j = [1, 1]$ for all j , the predicted first row is $x + 1$ in every column, so the SSE contribution is

$$(5 - (x + 1))^2 + (2 - (x + 1))^2 + (4 - (x + 1))^2 + (4 - (x + 1))^2 + (3 - (x + 1))^2 = (4 - x)^2 + (1 - x)^2 + (3 - x)^2 + (3 - x)^2.$$

Setting the derivative to zero:

$$-2[(4 - x) + (1 - x) + (3 - x) + (3 - x) + (2 - x)] = 0 \implies 13 - 5x = 0 \implies x = \frac{13}{5} = 2.6.$$

Next optimize v_{00} (upper-left of V), call it y . Only *column* 0 of the product depends on y . The users who rated item 0 are 0, 1, 3, 4, with ratings 5, 3, 2, 4 and user vectors $u_0 =$

$[2.6, 1]$, $u_1 = u_3 = u_4 = [1, 1]$, so the predicted first column is $2.6y + 1$, $y + 1$, $y + 1$, $y + 1$ and the SSE contribution is

$$(4 - 2.6y)^2 + (2 - y)^2 + (1 - y)^2 + (3 - y)^2.$$

Setting the derivative to zero:

$$-2[2.6(4 - 2.6y) + (2 - y) + (1 - y) + (3 - y)] = 0 \implies 16.4 - 9.76y = 0 \implies y = \frac{16.4}{9.76} \approx 1.68.$$

Only a row or a column is affected

When we change u_{ij} , only the predictions of row i change; when we change v_{ij} , only column j changes. This is why CGD can partition U by rows and V by columns and optimize each partition in parallel — the same parallelization argument ALS makes, but at the level of a single element.

19.3 The closed forms

CGD coordinate update for u_{ij}

$$u_{ij} = \frac{\sum_{s: r_{is} \neq \emptyset} v_{sj} \left(r_{is} - \sum_{t \neq j} u_{it} v_{st} \right)}{\sum_{s: r_{is} \neq \emptyset} v_{sj}^2}.$$

CGD coordinate update for v_{ij}

$$v_{ij} = \frac{\sum_{s: r_{si} \neq \emptyset} u_{sj} \left(r_{si} - \sum_{t \neq j} u_{st} v_{it} \right)}{\sum_{s: r_{si} \neq \emptyset} u_{sj}^2}.$$

Derivation of u_{ij} . The residual on user i 's rated item s is $r_{is} - \sum_t u_{it} v_{st} = r_{is} - u_{ij} v_{sj} - \sum_{t \neq j} u_{it} v_{st}$. The SSE over user i 's rated items is $\sum_s (r_{is} - u_{ij} v_{sj} - \sum_{t \neq j} u_{it} v_{st})^2$. Differentiating with respect to u_{ij} gives $\sum_s -2v_{sj} (r_{is} - u_{ij} v_{sj} - \sum_{t \neq j} u_{it} v_{st}) = 0$, so $\sum_s v_{sj} r_{is} - u_{ij} \sum_s v_{sj}^2 - \sum_s v_{sj} \sum_{t \neq j} u_{it} v_{st} = 0$, which rearranges to the formula. The v_{ij} derivation is symmetric (fix the item, sum over the users who rated it). $\square$

CGD — time

Updating one coordinate u_{ij} requires the inner “prediction without the j -th factor” $\sum_{t \neq j} u_{it} v_{st}$ for every item s user i rated, which is $O(|\Omega_i|d)$ using the code’s full-dot-product-minus-one-term trick. One full sweep updates all $nd + kd$ entries of U and V , so

$$O(|\Omega| d^2) \quad \text{per sweep.}$$

Each sweep is cheaper than one ALS iteration (no $O(d^3)$ solves), and unlike SGD each coordinate update is a closed-form optimum, giving CGD its “stable yet cheap” middle position between the two.

Source discrepancy — the v_{ij} summation condition

The *problem statement* (cell 13) writes the v_{ij} formula with the condition “ $r_{sj} \neq \emptyset$,” but the *derivation* (cell 14) and the *code* (cell 16) sum over *users s who rated item i* , i.e. “ $r_{si} \neq \emptyset$.” The condition $r_{sj} \neq \emptyset$ is a typo: it would iterate over a single user j ’s ratings instead of the users of item i . Use $r_{si} \neq \emptyset$ (the item i column) as in the box above.

The code’s “prediction without j ”

In `cd`, the inner term $\sum_{t \neq j} u_{it} v_{st}$ is computed as `np.dot(U[i,:], V[s,:]) - U[i,j]*V[s,j]`, i.e. the full dot product minus the j -th component. This avoids a loop over $t \neq j$ and is exactly the “prediction without the j -th factor” in the closed form.

19.4 Results and the storage advantage

On Jester with $k = 5$ latent factors and `tol=0.01`, `cd(df_jester, 5, 0.01)` returns 100×5 factor matrices whose first rows are asserted to be

$$F_{\text{user}}[0, :] = [-5.54176877, 1.11810181, 0.50455207, 2.50592182, 0.80730106],$$

$$F_{\text{item}}[0, :] = [1.00556239, 0.79076443, 1.48979126, 0.91302609, -0.71786162].$$

And `recommend_cd(0, ..., 10)` returns `[83, 73, 72, 80, 85, 81, 100, 98, 90, 51]`.

Storage: $(n + k)d \ll nk$

The factor matrices U ($n \times d$) and V ($k \times d$) together hold only $(n + k)d$ numbers, whereas the completed matrix has nk entries. For $n = k = 10^5$ users/items and $d = 50$, that is 10^7 numbers versus 10^{10} — three orders of magnitude fewer. Once U, V are learned, any user’s full rating vector is reconstructed (in parallel) as $u_i V^\top$, so we store the factors, not the completed matrix.

Chapter 20

The Surprise Library

Learning objectives

After this chapter you should be able to:

- explain each line of the `Surprise` usage (`KNNWithMeans`, `Reader`, `melt`, `load_from_df`);
- distinguish the full trainset, the anti-testset, and the testset;
- state how `Surprise`'s user-based CF with Pearson similarity relates to Part II's `user_complete`.

20.1 The code, line by line

The notebook verifies Part II's hand-written neighborhood code against the mature `Surprise` package (Problems 5 and 6):

```
1 from surprise import Reader, Dataset, KNNWithMeans
2
3 knn = KNNWithMeans(k=5, sim_options={"name": "pearson", "user_based":
4     True})
5 reader = Reader(rating_scale=(-10, 10))
6 df_melt = (
7     df_jester.reset_index()
8     .melt("index", var_name="itemID", value_name="rating")
9     .dropna()
10 )
11 dataset = Dataset.load_from_df(df_melt, reader)
12 knn.fit(dataset.build_full_trainset())
13 knn.predict(0, 1)
14 knn.test(knn.trainset.build_anti_testset())
15 knn.test(knn.trainset.build_testset());
```

Listing 20.1: `Surprise`: user-based CF with mean-centering

- `KNNWithMeans(k=5, sim_options={"name": "pearson", "user_based": True})` — a k -nearest-neighbor CF with *mean-centering*; `pearson` is exactly the adjusted cosine of Part II, and `user_based=True` selects user-user similarity.

- `Reader(rating_scale=(-10,10))` — tells Surprise the rating range so it can normalize correctly.
- `reset_index().melt(...).dropna()` — converts the wide 100×100 matrix into a long (user, itemID, rating) table and drops missing entries; **Surprise** requires this long format.
- `load_from_df` — builds a Surprise **Dataset** from the long frame.
- `build_full_trainset()` — train on *all* data (no hold-out), then `fit`.
- `predict(0,1)` — predict user 0's rating of item 1.
- `build_anti_testset()` — all *unrated* (user, item) pairs, used to *generate recommendations*;
- `build_testset()` — a held-out sample of *known* ratings, used to *evaluate prediction accuracy*.

anti-testset vs. testset

The *anti-testset* is the complement of the training ratings — the cells we do not know. We predict those to recommend. The *testset* is a withheld subset of the cells we *do* know, so we can compare predictions against truth. One is for producing recommendations, the other for measuring error.

Reconciling Surprise with your own code

`KNNWithMeans` with Pearson similarity and `user_based=True` is the library version of Part II's `user_complete` (mean-center rows, adjusted cosine over common items, weighted average of deviations). Setting `user_based=False` gives the item-based analogue for comparison (Problem 6).

Chapter 21

Summary: Formulas, Algorithms, and Exam Strategy

21.1 Formula sheet (Part III)

UV decomposition and objective

$$M \approx F_{\text{user}} F_{\text{item}}^\top, \quad J = \frac{1}{2} \sum_{(i,j): r_{ij} \neq \emptyset} (r_{ij} - \sum_s u_{is} v_{js})^2, \text{ with } e_{ij} = r_{ij} - \sum_s u_{is} v_{js}.$$

Gradient descent (batch)

$$u_{iq} \leftarrow u_{iq} + \alpha \sum_{j: r_{ij} \neq \emptyset} e_{ij} v_{jq}; \quad v_{jq} \leftarrow v_{jq} + \alpha \sum_{i: r_{ij} \neq \emptyset} e_{ij} u_{iq}; \quad \text{matrix form } U \leftarrow U + \alpha EV, V \leftarrow V + \alpha E^\top U.$$

SGD (single pair)

$$e_{ij} = r_{ij} - u_i \cdot v_j; \quad u_i \leftarrow u_i + \alpha e_{ij} v_j; \quad v_j \leftarrow v_j + \alpha e_{ij} u_i.$$

ALS row solve

$$u_i = \left(\sum_{j: r_{ij} \neq \emptyset} v_j v_j^\top \right)^{-1} \left(\sum_{j: r_{ij} \neq \emptyset} r_{ij} v_j \right) \text{ (and symmetric for } v_j); \text{ stop when RMSE} < \text{tol}.$$

CGD coordinate updates

$$u_{ij} = \frac{\sum_{s: r_{is} \neq \emptyset} v_{sj} (r_{is} - \sum_{t \neq j} u_{it} v_{st})}{\sum_{s: r_{is} \neq \emptyset} v_{sj}^2}; \quad v_{ij} = \frac{\sum_{s: r_{si} \neq \emptyset} u_{sj} (r_{si} - \sum_{t \neq j} u_{st} v_{it})}{\sum_{s: r_{si} \neq \emptyset} u_{sj}^2}.$$

21.2 Algorithm summaries and decision guide

- **Batch GD** — full-gradient step over all observed entries; matrix form $U \leftarrow U + \alpha EV$.
- **SGD** — per-entry gradient step; fastest, needs learning rate, noisy.
- **ALS** — alternate exact least-squares solves for all rows of V , then U ; stable, no learning rate, parallelizable.

- **CGD** — optimize one element at a time in closed form; balances stability and efficiency; parallelizable by row/column.
- Asked to *minimize a single coordinate*? Use the CGD closed form; with $d = 1$ the “ $\sum_{t \neq j}$ ” term vanishes.
- Asked to *solve one row exactly*? Use the ALS normal equations (or note that with all-ones U it collapses to a mean).
- Asked to *take one gradient step*? Distinguish batch (sum over a row/column) from SGD (one entry).
- Asked about *storage*? Quote $(n + k)d$ vs. nk .

21.3 Common exam traps and time savers

Most common mistakes

- Summing the objective over *all* cells instead of only observed ones ($r_{ij} \neq \emptyset$).
- Getting the SGD sign wrong: it is $+\alpha e_{ij} v_j$ (gradient descent subtracts a negative).
- Confusing the u_{ij} and v_{ij} summation conditions (user’s items vs. item’s users).
- Forgetting the $\frac{1}{2}$ in J (it cancels in the gradient, but matters if you are asked for the objective value).
- In the CGD worked example, updating the whole row instead of only the one element.

Time savers

- With all-ones U and d symmetric factors, each ALS item factor is just item-mean/ d ; no matrix solve needed.
- For $d = 1$, the CGD update is a simple ratio $\sum v_s r_s / \sum v_s^2$.
- The residual e_{ij} is reused verbatim in both the u_i and v_j SGD updates.
- Sanity check: a factor matrix times its transpose must reproduce observed ratings to within the RMSE you report.

Prerequisite reminders

- Least squares: minimizing $\sum_s (a_s - \beta x_s)^2$ over β gives $\beta = \sum x_s a_s / \sum x_s^2$.
- Chain rule on $\frac{1}{2}(f)^2$ gives $f \cdot f'$.
- Frobenius norm: $\|A\|_F^2 = \sum_{i,j} A_{ij}^2$.

Chapter 22

Practice Problems — Part III

Solutions and point-based rubrics are in Part IV (the answer key). The 5×5 matrix M used in the ALS and CGD examples (with empty cells at rows 2 and 4 as shown) is reused in several problems.

22.1 Category A — Conceptual

Exercise 22.1 (UV decomposition and latent factors). Category A — Conceptual Easy, 3 min Define the UV decomposition $M \approx F_{\text{user}} F_{\text{item}}^\top$. What are the shapes of the two factor matrices, what is the latent dimension d , and why does multiplying the factors “complete” the matrix?

Exercise 22.2 (Storage advantage). Category A — Conceptual Easy, 3 min If $n = k = 10^6$ and $d = 50$, how many numbers are needed to store (a) the completed utility matrix and (b) the two factor matrices? State the general $(n + k)d$ vs. nk comparison and its implication for storage.

Exercise 22.3 (Batch GD vs. SGD). Category A — Conceptual Moderate, 4 min Contrast batch gradient descent and stochastic gradient descent for the latent factor objective: what each update uses, the cost per update, and the stability/speed tradeoff.

Exercise 22.4 (ALS vs. SGD vs. CGD). Category A — Conceptual Moderate, 4 min Explain how ALS, SGD, and CGD each trade off stability against efficiency, and state the one feature ALS and SGD differ on most sharply (hint: learning rate).

Exercise 22.5 (Interpretation of latent factors). Category A — Conceptual Moderate, 4 min What is a “latent factor”? Explain why the factors are called *latent*, and why the model cannot attach a human-readable name to each factor automatically.

22.2 Category B — Basic computation

Exercise 22.6 (Objective value). Category B — Basic computation Easy, 4 min For $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$ (all observed), $d = 1$, $U = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $V = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, compute the objective $J = \frac{1}{2} \sum_{(i,j)} (r_{ij} - u_i v_j)^2$.

Exercise 22.7 (Gradient of one element). Category B — Basic computation Moderate, 5 min Using the same M, U, V as Exercise 22.6, compute $\partial J / \partial u_1$ (the single factor of user 0) and $\partial J / \partial v_1$ (the single factor of item 0).

Exercise 22.8 (One CGD coordinate ($d = 1$)). Category B — Basic computation Moderate, 5 min For $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$ with $d = 1$ and fixed $V = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, optimize user 0's factor u_1 by the CGD closed form. Then, fixing the new u_1 and $u_2 = 1$, optimize item 0's factor v_1 . Report both updated values.

22.3 Category C — Manual algorithm execution

Exercise 22.9 (ALS first iteration). Category C — Manual algorithm execution Moderate, 10 min On the 5×5 matrix M (with empty cells at row 2 = $\{-, -, 3, 1, 4\}$ and row 4 = $\{4, 4, 5, 4, -\}$), run the first ALS iteration with $d = 2$ and U initialized to all ones. Compute the five item factor vectors v_j (Step 1) and then verify the factor vector for *user 3* (rated $[2, 5, 4, 3, 5]$) in Step 2.

Exercise 22.10 (CGD first two coordinates). Category C — Manual algorithm execution Moderate, 10 min On the same 5×5 matrix with all-ones U and V ($d = 2$), optimize u_{00} and then v_{00} exactly as in the chapter, showing the derivative of each SSE. Report u_{00} and v_{00} .

Exercise 22.11 (SGD one epoch). Category C — Manual algorithm execution Moderate, 8 min For $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$, $d = 1$, initial $U = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $V = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\alpha = 0.1$, run one SGD epoch over the four observed entries in row-major order $(1, 1), (1, 2), (2, 1), (2, 2)$. Show every update and the final U, V .

22.4 Category D — Multi-step / integrated

Exercise 22.12 (Factorize and recommend). Category D — Multi-step / integrated Difficult, 15 min For $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$, $d = 1$, use CGD (full sweeps, tolerance 0.01) to converge to a rank-1 factorization $U = [u_1, u_2]^\top$, $V = [v_1, v_2]^\top$, then state the completed matrix $\hat{M} = UV^\top$ and explain why its rank is at most 1.

22.5 Category E — Challenging

Exercise 22.13 (Derive the u_{ij} closed form). Category E — Challenging Difficult, 10 min Starting from the SSE over user i 's rated items, $\sum_{s:r_{is} \neq 0} (r_{is} - u_{ij}v_{sj} - \sum_{t \neq j} u_{it}v_{st})^2$, differentiate with respect to u_{ij} , set to zero, and derive the closed form for u_{ij} .

Exercise 22.14 (All-ones ALS and the minimum norm). Category E — Challenging Difficult, 10 min Explain why, when U is initialized to all ones, the first ALS step for each item j reduces to “set each of the d factors to the item mean divided by d .” Relate your explanation to the fact that the normal-equation matrix $\sum_i u_i u_i^\top$ is rank-deficient for $d > 1$.

Part IV

Answer Key to Practice Problems

Chapter 23

Part I — Frequent Itemset Mining: Solutions

The grocery database used throughout Part I is $T_1 = \{\text{bread, butter, milk}\}$, $T_2 = \{\text{eggs, milk, yogurt}\}$, $T_3 = \{\text{bread, cheese, eggs, milk}\}$, $T_4 = \{\text{eggs, milk, yogurt}\}$, $T_5 = \{\text{cheese, milk, yogurt}\}$, with $N = 5$ and $\text{minsup} = 2$. The item support counts are $\text{sup}(\text{bread}) = 2$, $\text{sup}(\text{butter}) = 1$, $\text{sup}(\text{milk}) = 5$, $\text{sup}(\text{eggs}) = 3$, $\text{sup}(\text{yogurt}) = 3$, $\text{sup}(\text{cheese}) = 2$.

Ex. 1 — Support and relative support (A, Easy)

Solution. The *absolute support* of an itemset X is the number of transactions that contain X : $\text{sup}(X) = |\{T_i \in \mathcal{T} : X \subseteq T_i\}|$. The *relative support* is that count normalized by the total number of transactions N : $\text{relSup}(X) = \text{sup}(X)/N$. The two are related by a single division by N .

For $\{\text{bread, milk}\}$, the transactions containing both items are T_1 and T_3 , so $\text{sup}(\{\text{bread, milk}\}) = 2$ and

$$\text{relSup}(\{\text{bread, milk}\}) = \frac{2}{5} = 0.4.$$

Verification: $0.4 \cdot N = 0.4 \cdot 5 = 2 = \text{sup}$, consistent.

Rubric (2 pts) correct definitions of absolute and relative support; (1 pts) correct conversion; (1 pts) $\text{sup} = 2$; (1 pts) $\text{relSup} = 0.4$.

Common mistake. Reporting $\text{relSup} = 2$ (absolute) instead of $2/5$, or writing the ratio upside down (N/sup).

Ex. 2 — Monotonicity and pruning (A, Easy)

Solution. *Support monotonicity (downward closure):* if $Y \subseteq X$ then $\text{sup}(Y) \geq \text{sup}(X)$. Adding items to an itemset can only shrink the set of transactions that contain it, so support never increases.

It is safe to discard a candidate X whenever *any* proper subset $Y \subsetneq X$ is infrequent: by monotonicity, $\text{sup}(X) \leq \text{sup}(Y) < \text{minsup}$, so X itself cannot possibly be frequent. Testing the (already computed) subsets costs nothing compared to scanning the database for X , which is exactly what makes the pruning rule correct and cheap.

Rubric (2 pts) correct statement ($Y \subseteq X \Rightarrow \text{sup}(Y) \geq \text{sup}(X)$); (2 pts) correct safety argument (a subset's infrequency forces the superset's infrequency).

Common mistake. Stating the implication backwards ($\text{sup}(X) \geq \text{sup}(Y)$), which would make pruning unsafe.

Ex. 3 — Frequent vs. maximal (A, Moderate)

Solution. A *frequent* itemset is one whose support meets *minsup*. A *maximal* frequent itemset is a frequent itemset with *no frequent proper superset* — it sits at the top of a chain of frequent sets.

A maximal frequent itemset can *and must* contain frequent proper subsets: by downward closure every subset of a frequent itemset is frequent, so the subsets of a maximal set are automatically frequent. A frequent itemset that has a frequent proper superset is *not* maximal (that superset witnesses its non-maximality). Maximality is about there being no larger frequent set, not about the contents being “unusual.”

In the grocery database the maximal frequent itemsets are {bread, milk}, {cheese, milk}, and {eggs, milk, yogurt}: each is frequent and no frequent strict superset exists, whereas e.g. {milk, eggs} is frequent but not maximal because {milk, eggs, yogurt} is frequent.

Rubric (2 pts) correct definitions; (2 pts) both sub-questions answered correctly with justification; (1 pts) correct maximal sets for the database.

Ex. 4 — Lift interpretation (A, Easy)

Solution. $\text{lift}(A \rightarrow B) = \text{conf}(A \rightarrow B) / \text{relSup}(B)$. It measures how much more likely B becomes once we know A is present, compared to B 's baseline popularity.

- lift > 1: A and B co-occur *more* often than chance — positive association.
- lift = 1: independence — A tells us nothing about B .
- lift < 1: negative association — A and B repel each other.

Lift is preferred over confidence because confidence ignores how common the consequent already is. A rule can have high confidence purely because B is popular everywhere (e.g. $\text{conf} \approx 0.9$ when $\text{relSup}(B) = 0.9$); lift divides out that baseline, so only a rule that actually *raises* the probability of B scores above 1.

Rubric (1 pts) formula; (3 pts) three interpretations; (2 pts) why lift beats confidence (baseline normalization).

Ex. 5 — Support computations (B, Easy)

Solution.

- $\text{sup}(\{\text{eggs}, \text{milk}\})$: both in $T_2, T_3, T_4 \Rightarrow \mathbf{3}$.
- $\text{sup}(\{\text{bread}, \text{cheese}\})$: both only in $T_3 \Rightarrow \mathbf{1}$ (infrequent).
- $\text{relSup}(\{\text{milk}\}) = 5/5 = \mathbf{1}$ (milk is in every transaction).
- $\text{sup}(\{\text{eggs}, \text{milk}, \text{yogurt}\})$: both in T_2 and $T_4 \Rightarrow \mathbf{2}$.

Verification: the maximum support of any itemset is sup of its rarest item; $\{\text{bread}, \text{cheese}\}$ cannot exceed $\min(2, 2) = 2$ and in fact is 1, and $\{\text{eggs}, \text{milk}, \text{yogurt}\}$ cannot exceed $\min(3, 5, 3) = 3$ and is 2.

Rubric (1 pts) each of the four values.

Common mistake. Forgetting the sentinel: counting $\{\text{bread}, \text{cheese}\} = 2$ by including a transaction that has only one of them.

Ex. 6 — Confidence and lift (B, Moderate)

Solution. Given $N = 20$, $\text{sup}(A) = 10$, $\text{sup}(B) = 8$, $\text{sup}(A \cup B) = 4$:

$$\text{relSup}(B) = \frac{8}{20} = 0.4, \quad \text{conf}(A \rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)} = \frac{4}{10} = 0.4,$$

$$\text{lift}(A \rightarrow B) = \frac{\text{conf}(A \rightarrow B)}{\text{relSup}(B)} = \frac{0.4}{0.4} = 1.0.$$

A lift of exactly 1 means A and B are *independent*: knowing A does not change the probability of B ($\text{conf} = 0.4 = \text{relSup}(B)$). The rule carries no information despite a non-trivial confidence value.

Rubric (1 pts) $\text{relSup}(B)$; (1 pts) confidence; (1 pts) lift; (1 pts) interpretation (= 1 means independence).

Common mistake. Confusing confidence and lift, or computing lift as $\text{conf} / \text{sup}(B)$ (forgetting to divide $\text{sup}(B)$ by N), which would give $0.4/8 = 0.05$.

Ex. 7 — Apriori by hand (C, Moderate)

Solution. Database $T_1 = \{a, b, c\}$, $T_2 = \{a, b, d\}$, $T_3 = \{a, c\}$, $T_4 = \{b, c, d\}$, $T_5 = \{a, b\}$, $\text{minsup} = 2$.

Pass 1 (F_1). Count single items: a : $T_1, T_2, T_3, T_5 \Rightarrow \mathbf{4}$; b : $T_1, T_2, T_4, T_5 \Rightarrow \mathbf{4}$; c : $T_1, T_3, T_4 \Rightarrow \mathbf{3}$; d : $T_2, T_4 \Rightarrow \mathbf{2}$. All ≥ 2 , so $F_1 = \{a : 4, b : 4, c : 3, d : 2\}$.

Pass 2 (C_2, F_2). Candidate pairs from F_1 : $\{ab, ac, ad, bc, bd, cd\}$ (six pairs). Count each:

pair	ab	ac	ad	bc	bd	cd
support	3	2	1	2	2	1

(ab : T_1, T_2, T_5 ; ac : T_1, T_3 ; ad : T_2 only; bc : T_1, T_4 ; bd : T_2, T_4 ; cd : T_4 only.) Hence $F_2 = \{ab : 3, ac : 2, bc : 2, bd : 2\}$.

Pass 3 (C_3 , pruning, F_3). Generate triples from F_2 and prune those with an infrequent 2-subset:

- abc : subsets ab, ac, bc all frequent $\Rightarrow$ *survives*;
- abd : subset ad infrequent $\Rightarrow$ *pruned*;
- acd : subsets ad, cd infrequent $\Rightarrow$ *pruned*;
- bcd : subset cd infrequent $\Rightarrow$ *pruned*.

So $C_3 = \{abc\}$. Counting, abc appears only in T_1 , so $\text{sup}(abc) = 1 < 2 \Rightarrow F_3 = \emptyset$. Apriori stops.

All frequent itemsets (8 total). $\{a\} : 4, \{b\} : 4, \{c\} : 3, \{d\} : 2, \{ab\} : 3, \{ac\} : 2, \{bc\} : 2, \{bd\} : 2$.

Maximal. The four frequent pairs $\{ab, ac, bc, bd\}$ have no frequent superset (no frequent triple exists), so *all four* are maximal; no singleton is maximal (each has a frequent superset).

Rubric (2 pts) F_1 ; (2 pts) C_2 and F_2 ; (2 pts) C_3 with pruning and F_3 ; (1 pts) full list; (1 pts) maximal set.

Common mistake. Counting $ad = 2$ (it is 1) or failing to prune abc , both of which change the maximal answer.

Ex. 8 — Association rules by hand (C, Moderate)

Solution. From Ex. 7 the frequent itemsets are $\{a\} : 4, \{b\} : 4, \{c\} : 3, \{d\} : 2, \{ab\} : 3, \{ac\} : 2, \{bc\} : 2, \{bd\} : 2$; and $N = 5$. Single-item-consequent rules come from the four size-2 itemsets. With $\text{minconf} = 0.5$, and $\text{relSup}(a) = 0.8, \text{relSup}(b) = 0.8, \text{relSup}(c) = 0.6, \text{relSup}(d) = 0.4$:

rule	conf	lift	support	kept?
$a \rightarrow b$	$3/4 = 0.75$	$0.75/0.8 = 0.9375$	3	yes
$b \rightarrow a$	$3/4 = 0.75$	$0.75/0.8 = 0.9375$	3	yes
$a \rightarrow c$	$2/4 = 0.5$	$0.5/0.6 = 0.8333$	2	yes
$c \rightarrow a$	$2/3 = 0.667$	$0.667/0.8 = 0.8333$	2	yes
$b \rightarrow c$	$2/4 = 0.5$	$0.5/0.6 = 0.8333$	2	yes
$c \rightarrow b$	$2/3 = 0.667$	$0.667/0.8 = 0.8333$	2	yes
$b \rightarrow d$	$2/4 = 0.5$	$0.5/0.4 = 1.25$	2	yes
$d \rightarrow b$	$2/2 = 1.0$	$1.0/0.8 = 1.25$	2	yes

All eight rules meet *minconf* (none discarded). Sorting by (decreasing lift, decreasing confidence, decreasing support), the two lift-1.25 rules come first: $d \rightarrow b$ (confidence 1.0) then $b \rightarrow d$ (confidence 0.5). So the **top two** are $d \rightarrow b$ and $b \rightarrow d$.

Verification: $d \rightarrow b$ has $\text{conf} = \sup(bd)/\sup(d) = 2/2 = 1.0$ — every purchase of d is accompanied by b — and $\text{lift} = 1.0/(4/5) = 1.25 > 1$, a genuinely positive association.

Rubric **(3 pts)** all 8 rules with correct confidence; **(3 pts)** correct lift values; **(2 pts)** correct top-two ordering.

Common mistake. Alphabetizing ties instead of using the stable confidence/support order, or treating $d \rightarrow b$ and $b \rightarrow d$ as interchangeable.

Ex. 9 — FP-Growth by hand (C, Difficult)

Solution.

(a) Frequent items in decreasing support. Dropping butter (support 1): milk 5, eggs 3, yogurt 3, bread 2, cheese 2. Breaking ties alphabetically, the *header order* is milk > eggs > yogurt > bread > cheese.

(b) Sorted transactions and the FP-tree. Sort each transaction by that order (dropping infrequent items):

$$\begin{aligned}
 T_1 &\mapsto [\text{milk}, \text{bread}], & T_2 &\mapsto [\text{milk}, \text{eggs}, \text{yogurt}], \\
 T_3 &\mapsto [\text{milk}, \text{eggs}, \text{bread}, \text{cheese}], & T_4 &\mapsto [\text{milk}, \text{eggs}, \text{yogurt}], \\
 T_5 &\mapsto [\text{milk}, \text{yogurt}, \text{cheese}].
 \end{aligned}$$

Inserting these builds the FP-tree:

```

root
  |-- milk:5
    |-- bread:1
    |-- eggs:3
    |   |-- yogurt:2
    |   |-- bread:1
    |   |-- cheese:1
    |-- yogurt:1
    |-- cheese:1

```

(c) **Header table.** In decreasing-support order, with node links:

- milk (5): one node `milk:5`;
- eggs (3): one node `eggs:3` (under milk);
- yogurt (3): two nodes `yogurt:2` (under eggs) and `yogurt:1` (under milk);
- bread (2): two nodes `bread:1` (under milk) and `bread:1` (under eggs);
- cheese (2): two nodes `cheese:1` (under eggs→bread) and `cheese:1` (under milk→yogurt).

(d) **Ascending-support recursion.** Mine in the order `cheese → bread → yogurt → eggs → milk` (least frequent first).

cheese (2): paths `milk-eggs-bread-cheese` and `milk-yogurt-cheese`; conditional pattern base $\{\text{milk, eggs, bread}\} : 1, \{\text{milk, yogurt}\} : 1$. Only milk (2) is frequent $\Rightarrow \{\text{cheese, milk}\} : 2$.

bread (2): paths `milk-bread` and `milk-eggs-bread`; base $\{\text{milk}\} : 1, \{\text{milk, eggs}\} : 1$. Only milk (2) is frequent $\Rightarrow \{\text{bread, milk}\} : 2$.

yogurt (3): paths `milk-eggs-yogurt` (twice) and `milk-yogurt`; base $\{\text{milk, eggs}\} : 2, \{\text{milk}\} : 1$. Both milk (3) and eggs (2) are frequent; the conditional tree is `milk:3-eggs:2`, which recurses to $\{\text{yogurt, milk}\} : 3, \{\text{yogurt, eggs}\} : 2$, and $\{\text{yogurt, milk, eggs}\} : 2$.

eggs (3): path `milk-eggs` (thrice); base $\{\text{milk}\} : 3 \Rightarrow \{\text{eggs, milk}\} : 3$.

milk (5): no prefix (top of tree) $\Rightarrow \{\text{milk}\} : 5$.

Result. The eleven frequent itemsets, $\{\text{milk}\} : 5, \{\text{eggs}\} : 3, \{\text{yogurt}\} : 3, \{\text{bread}\} : 2, \{\text{cheese}\} : 2$, and the pairs/triple $\{\text{cheese, milk}\} : 2, \{\text{bread, milk}\} : 2, \{\text{yogurt, milk}\} : 3, \{\text{yogurt, eggs}\} : 2, \{\text{eggs, milk}\} : 3, \{\text{yogurt, milk, eggs}\} : 2$, matching Apriori. Maximal: $\{\text{bread, milk}\}, \{\text{cheese, milk}\}, \{\text{eggs, milk, yogurt}\}$.

Rubric (1 pts) (a); (3 pts) (b) correct tree; (2 pts) (c); (4 pts) (d) all conditional bases and resulting itemsets.

Common mistake. Sorting the tree in ascending order, or failing to remove butter before building the tree, which corrupts every conditional count.

Ex. 10 — Full FIM → ARM pipeline (D, Difficult)

Solution. Combining Ex. 7 and Ex. 8: the frequent itemsets (Ex. 7) yield the eight association rules with confidence and lift (Ex. 8). After sorting, the top rule is $d \rightarrow b$ with $\text{conf} = 1.0$ and $\text{lift} = 1.25$.

Business interpretation: “Every customer who buys item d also buys item b (confidence 100%), and this co-occurrence is 25% stronger than chance (lift 1.25).” Actionable: place b near d , or recommend b whenever d is in the basket.

Rubric (4 pts) correct frequent itemsets; (4 pts) correct rules with conf/lift; (2 pts) correct sort; (2 pts) sensible interpretation.

Ex. 11 — Prove monotonicity (E, Moderate)

Solution. Let $\mathcal{T}(X) = \{T_i : X \subseteq T_i\}$ be the set of transactions containing X . If $Y \subseteq X$, then every transaction that contains X also contains Y (since X already contains all of Y 's items): $\mathcal{T}(X) \subseteq \mathcal{T}(Y)$. Taking cardinalities, $\text{sup}(X) = |\mathcal{T}(X)| \leq |\mathcal{T}(Y)| = \text{sup}(Y)$.

Downward closure follows immediately: if X is frequent ($\text{sup}(X) \geq \text{minsup}$) then every subset Y satisfies $\text{sup}(Y) \geq \text{sup}(X) \geq \text{minsup}$, so every subset is frequent. Contrapositively, if Y is infrequent then no superset of Y can be frequent — the Apriori pruning rule.

Rubric (3 pts) clean proof ($\mathcal{T}(X) \subseteq \mathcal{T}(Y)$ argument); (1 pts) derivation of downward closure.

Common mistake. Assuming monotonicity rather than proving it, or only proving the contrapositive without establishing the set-containment fact.

Ex. 12 — Worst-case analysis (E, Difficult)

Solution. Apriori's pruning only eliminates candidates whose *some subset* is infrequent. If *every* non-empty itemset is frequent, then no candidate is ever pruned, and Apriori enumerates all $2^d - 1$ non-empty subsets of the universe — the same as brute force.

Concrete worst case: one transaction containing all d items, $\mathcal{T} = \{\{1, 2, \dots, d\}\}$, with $\text{minsup} = 1$. Then every non-empty itemset has support $1 \geq 1$, so $F_1, \dots, F_d$ are all maximal, and Apriori generates every level in full: $\binom{d}{1} + \binom{d}{2} + \dots + \binom{d}{d} = 2^d - 1$ candidates.

Pruning is *most effective* when the data is sparse and minsup is high, so many itemsets are infrequent early and entire branches of the lattice are cut. It is *useless* when the data is dense and minsup is low (as above), where nearly all subsets are frequent. In general the number of frequent itemsets can itself be exponential in d , so no algorithm that outputs them all can beat exponential time in the worst case.

Rubric (3 pts) why pruning can fail (all subsets frequent); (3 pts) concrete database + minsup ; (2 pts) when pruning helps vs. hurts.

Ex. 13 — Choosing thresholds (E, Moderate)

Solution. (i) Lowering minsup makes more itemsets qualify as frequent; each new frequent itemset of size ≥ 2 also becomes a source of additional association rules. Both the number of discovered itemsets and the number of rules grow, often dramatically (long-tail itemsets appear), and the result is noisier and harder to interpret.

(ii) DOTCOM POSTAGE appears in a huge fraction of baskets because it is a *shipping artifact* appended to many orders, not merchandise. Its extreme support inflates the confidence and support of any rule involving it, so it dominates the top of the rule list by the (lift, confidence, support) sort. It is excluded from merchandising recommendations because a shipping line cannot be bundled or cross-sold the way a physical product can — recommending postage is not a useful merchandising action.

Rubric **(2 pts)** (i) growth of itemsets and rules; **(3 pts)** (ii) why it dominates and why it is excluded.

Chapter 24

Part II — Neighborhood-based Collaborative Filtering: Solutions

The 4×7 `df_utility` matrix (code-authoritative) is

User	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

with user means $\mu_A = \frac{10}{3}$, $\mu_B = \frac{14}{3}$, $\mu_C = \frac{11}{3}$, $\mu_D = 3$.

Ex. 1 — Utility matrix and rating types (A, Easy)

Solution. A *utility matrix* has one row per user and one column per item; entry r_{ui} records user u 's rating of item i (or is empty/unknown). *Explicit* ratings are directly given by users (stars, scores); *implicit* ratings are inferred from behavior (purchases, clicks, time spent). The matrix is almost always sparse because each user rates only a tiny fraction of the catalog, so most cells are empty.

Rubric (2 pts) definition; (2 pts) explicit vs implicit; (1 pts) sparsity.

Ex. 2 — Content-based vs. collaborative (A, Easy)

Solution. *Content-based* filtering uses the *features of the items* (genre, keywords, author) and the user's own history to recommend similar items. *Collaborative* filtering uses only the *ratings themselves* — the opinions of other users — ignoring item content.

For jokes (whose content is hard to describe in features), collaborative filtering needs no item description: it exploits “users who laughed at the same jokes as you also liked this one,” which works even when we cannot say *why* a joke is funny. (Notebook: the Jester dataset is exactly this situation.)

Rubric (2 pts) contrast; (1 pts) advantage.

Ex. 3 — Neighborhood vs. model-based; user vs. item (A, Moderate)

Solution. *Memory-based (neighborhood)* CF computes recommendations directly from stored ratings using similarity between rows or columns, with no learned model. *Model-based* CF learns a compact model (e.g. a low-rank factorization) from the data and predicts with it. *User-based* CF finds similar *users*; *item-based* CF finds similar *items*.

The user/item duality breaks for two reasons (notebook cell 26): (1) with similar *users* you can recommend immediately, whereas similar *items* still need an extra aggregation step over a user’s rated items; (2) items cluster into *genres*, so item similarity is well-defined, while two users may overlap in only one genre and differ elsewhere, making user similarity less reliable.

Rubric (2 pts) memory vs model; (2 pts) user vs item; (2 pts) two reasons.

Ex. 4 — Why mean-center? (A, Moderate)

Solution. Mean-centering removes each user’s personal rating *baseline*, so similarity measures *agreement relative to one’s own average* rather than magnitude. Without it, a user who rates everything 5 and a user who rates everything 3 have *perfectly aligned* raw vectors and raw cosine 1, yet they express very different enthusiasm — raw cosine confuses “same direction” with “same opinion.” After centering, the all-5 user and all-3 user both become the zero vector (similarity undefined/0), correctly reflecting that we learned nothing about their *relative* preferences.

Concrete pair: $u = [5, 5, 5]$, $v = [3, 3, 3]$. Raw cosine is $\frac{45}{\sqrt{75}\sqrt{27}} = 1$, but centered both are $[0, 0, 0]$ and the adjusted cosine is 0 (undefined).

Rubric (2 pts) why centering; (2 pts) why raw cosine is poor; (1 pts) concrete pair.

Ex. 5 — Raw cosine (B, Easy)

Solution.

- $\text{RawCosine}(B, D)$: common item {HP2} only. Numerator $5 \cdot 3 = 15$; denominator $\sqrt{5^2}\sqrt{3^2} = 5 \cdot 3 = 15$; result $15/15 = 1$.
- $\text{RawCosine}(B, C)$: B rated {HP1, HP2, HP3}, C rated {TW, SW1, SW2}; common set is empty, so the denominator is 0 and the function returns 0.

Rubric (1 pts) common sets; (2 pts) B,D arithmetic; (1 pts) B,C = 0.

Common mistake. Computing over *all* items (treating empty cells as 0) instead of common items only.

Ex. 6 — Adjusted cosine (B, Moderate)

Solution. Using the centered values (A: HP1 = $\frac{2}{3}$, TW = $\frac{5}{3}$, SW1 = $-\frac{7}{3}$; B: HP1 = $\frac{1}{3}$, HP2 = $\frac{1}{3}$, HP3 = $-\frac{2}{3}$; C: TW = $-\frac{5}{3}$, SW1 = $\frac{1}{3}$, SW2 = $\frac{4}{3}$; D: HP2 = 0, SW3 = 0):

- $\text{sim}(A, C)$, common {TW, SW1}:

$$\frac{\frac{5}{3}(-\frac{5}{3}) + (-\frac{7}{3})(\frac{1}{3})}{\sqrt{(\frac{5}{3})^2 + (-\frac{7}{3})^2} \sqrt{(-\frac{5}{3})^2 + (\frac{1}{3})^2}} = \frac{-\frac{25}{9} - \frac{7}{9}}{\sqrt{\frac{74}{9}} \sqrt{\frac{26}{9}}} = \frac{-\frac{32}{9}}{\frac{\sqrt{1924}}{9}} = -\frac{32}{\sqrt{1924}} \approx -0.7295.$$

- $\text{sim}(B, D)$, common {HP2}: numerator $(\frac{1}{3})(0) = 0$; denominator $(\frac{1}{3})(0) = 0$; result **0** (empty/zero denominator).

Rubric **(2 pts)** $\text{sim}(A, C)$ exact fraction; **(1 pts)** decimal; **(1 pts)** $\text{sim}(B, D)$.

Common mistake. Forgetting to center, or using the raw (signed) denominator instead of the norm product.

Ex. 7 — Regression metrics (B, Easy)

Solution. $r = [1, 4, 2, 5]$, $\hat{r} = [2, 4, 1, 4]$; errors $\hat{r} - r = [1, 0, -1, -1]$.

$$\text{MAE} = \frac{1 + 0 + 1 + 1}{4} = \frac{3}{4} = \mathbf{0.75}, \quad \text{MSE} = \frac{1 + 0 + 1 + 1}{4} = \frac{3}{4} = \mathbf{0.75},$$

$$\text{RMSE} = \sqrt{0.75} \approx \mathbf{0.8660}.$$

The three non-exact predictions (items 1, 3, 4) each contribute $1^2 = 1$ to the MSE sum and therefore *tie* for largest contribution; the second prediction (exact) contributes 0.

Rubric **(1 pts)** errors; **(1 pts)** MAE; **(1 pts)** MSE; **(1 pts)** RMSE; **(1 pts)** largest contributor (three-way tie).

Ex. 8 — DCG of one user (B, Moderate)

Solution. True ratings $\{x : 5, y : 3, z : 2, w : 4\}$; recommendation $[x, y, w]$ with $L = 3$.

$$\text{DCG} = \frac{2^5 - 1}{\log_2 2} + \frac{2^3 - 1}{\log_2 3} + \frac{2^4 - 1}{\log_2 4} = 31 + \frac{7}{1.585} + \frac{15}{2} = 31 + 4.416 + 7.5 = \mathbf{42.916}.$$

The ideal top-3 is the user's own ratings descending: $x = 5, w = 4, y = 3$:

$$\text{IDCG} = 31 + \frac{15}{\log_2 3} + \frac{7}{2} = 31 + 9.464 + 3.5 = \mathbf{43.964}.$$

$$\text{NDCG} = \frac{42.916}{43.964} \approx \mathbf{0.976}.$$

The system is near-ideal: it placed the best item first and only swapped w and y in ranks 2 and 3.

Rubric **(2 pts)** DCG terms; **(2 pts)** IDCG terms; **(1 pts)** NDCG; **(1 pts)** interpretation.

Common mistake. Building IDCG from the recommendation order instead of the true-rating order, or using $\log_{10}$ instead of $\log_2$.

Ex. 9 — User-based CF by hand (C, Moderate)

Solution. B's similarities: $\text{sim}(B, A) = 1$, $\text{sim}(B, C) = 0$ (no common item), $\text{sim}(B, D) = 0$ (common HP2 but D is centered to 0). Ordering by $(-\text{sim}, \text{id})$: $A(-1), C(0), D(0)$; top $k = 2$ are $N = \{A, C\}$. Missing items: TW, SW1, SW2, SW3.

- **TW** (A rated 5, centered $\frac{5}{3}$; C rated 2, centered $-\frac{5}{3}$): $\text{num} = 1 \cdot \frac{5}{3} + 0 \cdot (-\frac{5}{3}) = \frac{5}{3}$; $\text{den} = |1| + |0| = 1$; $\hat{r} = \frac{14}{3} + \frac{5}{3} = \frac{19}{3} \approx \mathbf{6.333}$.
- **SW1** (A 1, centered $-\frac{7}{3}$; C 4, centered $\frac{1}{3}$): $\text{num} = 1 \cdot (-\frac{7}{3}) + 0 \cdot \frac{1}{3} = -\frac{7}{3}$; $\text{den} = 1$; $\hat{r} = \frac{14}{3} - \frac{7}{3} = \frac{7}{3} \approx \mathbf{2.333}$.
- **SW2** (only C rated, $\text{sim} = 0$): $\text{num} = 0$, $\text{den} = |0| = 0 \Rightarrow \mathbf{NaN}$.
- **SW3** (no neighbor rated it): $\Rightarrow \mathbf{NaN}$.

B's completed row: $[5, 5, 4, \frac{19}{3}, \frac{7}{3}, \text{NaN}, \text{NaN}]$.

Rubric (2 pts) similarities; (2 pts) neighbor ordering; (2 pts) TW/SW1 arithmetic; (1 pts) NaN cells.

Common mistake. Re-selecting neighbors per item instead of fixing N once, which would pull in user D for some cells.

Ex. 10 — Item-based CF by hand (C, Moderate)

Solution. HP2's item neighbors (from the chapter): $\text{sim}(\text{HP2}, \text{HP1}) = 1$ (common user B), $\text{sim}(\text{HP2}, \text{HP3}) = -1$ (common user B), all others 0. Ordering by $(-\text{sim}, \text{id})$: $\text{HP1}(-1), \text{TW}(0), \text{SW1}(0), \text{SW2}(0), \text{SW3}(0), \text{HP3}(1)$; top $k = 2$ are $\{\text{HP1}, \text{TW}\}$.

User A rated $\text{HP1} = 4$ (centered $\frac{2}{3}$) and $\text{TW} = 5$ (centered $\frac{5}{3}$):

$$\text{num} = \underbrace{1 \cdot \frac{2}{3}}_{\text{HP1}} + \underbrace{0 \cdot \frac{5}{3}}_{\text{TW}} = \frac{2}{3}, \quad \text{den} = |1| + |0| = 1,$$

$$\hat{r}_{A, \text{HP2}} = \frac{10}{3} + \frac{2}{3} = \frac{12}{3} = \mathbf{4.0}.$$

Rubric (2 pts) item similarities; (2 pts) neighbor ordering; (2 pts) num/den; (1 pts) final value.

Common mistake. Centering by item (column) instead of user (row) before computing item-item similarity.

Ex. 11 — NDCG by hand (C, Moderate)

Solution. With $L = 2$ and $1/\log_2 3 \approx 0.63093$:

User 1 ($\{a : 5, b : 2, c : 4\}$, rec $[c, a]$): $\text{DCG}_1 = \frac{2^4-1}{\log_2 2} + \frac{2^5-1}{\log_2 3} = 15 + 31(0.63093) = 15 + 19.559 = \mathbf{34.559}$. Ideal $[a, c]$: $\text{IDCG}_1 = \frac{31}{1} + \frac{15}{\log_2 3} = 31 + 9.464 = \mathbf{40.464}$.

User 2 ($\{a : 1, b : 5, c : 2\}$, rec $[b, a]$): $\text{DCG}_2 = \frac{2^5-1}{1} + \frac{2^1-1}{\log_2 3} = 31 + 1(0.63093) = 31 + 0.631 = \mathbf{31.631}$. Ideal $[b, c]$: $\text{IDCG}_2 = \frac{31}{1} + \frac{2^2-1}{\log_2 3} = 31 + 3(0.63093) = 31 + 1.893 = \mathbf{32.893}$.

DCG over both users (the *mean*) = $\frac{34.559+31.631}{2} = \frac{66.190}{2} = \mathbf{33.095}$. NDCG (the *global ratio*) = $\frac{34.559+31.631}{40.464+32.893} = \frac{66.190}{73.357} \approx \mathbf{0.902}$.

Rubric (2 pts) each user's DCG; (2 pts) each user's IDCG; (1 pts) mean DCG; (1 pts) global-ratio NDCG.

Common mistake. Averaging the two per-user NDCGs ($\frac{0.854+0.962}{2}$) instead of taking the ratio of the sums, which gives a different (wrong) value.

Ex. 12 — Full pipeline and recommendation (D, Difficult)

Solution. Pairwise adjusted cosines: $\text{sim}(A, B) = 1$, $\text{sim}(A, C) = -32/\sqrt{1924} \approx -0.7295$, $\text{sim}(A, D) = 0$, $\text{sim}(B, C) = 0$, $\text{sim}(B, D) = 0$, $\text{sim}(C, D) = 0$.

Neighbors ($k = 2$, by $(-\text{sim}, \text{id})$): $A \rightarrow \{B, D\}$; $B \rightarrow \{A, C\}$; $C \rightarrow \{B, D\}$; $D \rightarrow \{A, B\}$.

User A (Ex. 9 in the chapter): $\text{HP2} = \frac{11}{3}$, $\text{HP3} = \frac{8}{3}$, $\text{SW2} = \text{NaN}$, $\text{SW3} = \text{NaN} \Rightarrow A = [4, \frac{11}{3}, \frac{8}{3}, 5, 1, \text{NaN}, \text{NaN}]$.

User B (Ex. 9 above): $\text{TW} = \frac{19}{3}$, $\text{SW1} = \frac{7}{3}$, $\text{SW2} = \text{NaN}$, $\text{SW3} = \text{NaN} \Rightarrow B = [5, 5, 4, \frac{19}{3}, \frac{7}{3}, \text{NaN}, \text{NaN}]$.

User C with neighbors $\{B, D\}$ (both $\text{sim} = 0$): every candidate has $\text{den} = 0$, so HP1 , HP2 , HP3 , SW3 are all **NaN** $\Rightarrow C = [\text{NaN}, \text{NaN}, \text{NaN}, 2, 4, 5, \text{NaN}]$.

User D with neighbors $\{A, B\}$ (both $\text{sim} = 0$): HP1 , HP3 , TW , SW1 , SW2 all **NaN** $\Rightarrow D = [\text{NaN}, 3, \text{NaN}, \text{NaN}, \text{NaN}, \text{NaN}, 3]$.

Completed matrix:

User	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4	$\frac{11}{3}$	$\frac{8}{3}$	5	1	NaN	NaN
B	5	5	4	$\frac{19}{3}$	$\frac{7}{3}$	NaN	NaN
C	NaN	NaN	NaN	2	4	5	NaN
D	NaN	3	NaN	NaN	NaN	NaN	3

Recommend A (top-2): unrated items HP2, HP3, SW2, SW3 have predictions $\frac{11}{3} \approx 3.667$, $\frac{8}{3} \approx 2.667$, NaN, NaN; drop NaN; sort by rating desc $\Rightarrow [\text{HP2}, \text{HP3}]$.

Recommend B (top-2): unrated TW, SW1, SW2, SW3 have predictions $\frac{19}{3} \approx 6.333$, $\frac{7}{3} \approx 2.333$, NaN, NaN $\Rightarrow [\text{TW}, \text{SW1}]$.

Rubric (3 pts) similarity matrix; (2 pts) neighbor sets; (4 pts) completed matrix; (2 pts) top-2 for A and B with justification.

Common mistake. Not dropping NaN before sorting, or forgetting the stable id-first sort (which matters when predicted ratings tie).

Ex. 13 — Coverage and cold start (E, Moderate)

Solution. Coverage is the fraction of users who can be given n recommendations (equivalently, who receive at least one prediction). Under user-based CF with $k = 2$, users C and D receive no predictions: C's top-2 neighbors $\{B, D\}$ both have similarity 0, and D's top-2 neighbors $\{A, B\}$ both have similarity 0. Every candidate's denominator $\sum |\text{sim}|$ is 0, so every missing cell stays NaN and neither user can be recommended anything.

This is the neighborhood analogue of the *cold-start* problem: a user with no (or only zero-similarity) overlap with the population cannot be served by a similarity-based method, which lowers coverage. Cold start is the extreme case — a brand-new user has no ratings at all, hence no similar peers.

Rubric (2 pts) coverage definition; (2 pts) identify C and D and why; (2 pts) link to cold start.

Ex. 14 — The abs denominator (E, Difficult)

Solution. (a) With the unsigned denominator $\sum_v \text{sim}(u, v)$, a peer group with *mixed or negative* similarities can have a sum near zero or even negative, making the prediction $\mu_u + \text{num}/\text{den}$ blow up in magnitude or flip sign. Since each term $\text{sim}(u, v)(r_{vi} - \mu_v)$ already carries the direction of the neighbor, the denominator should measure *total weight*, which is inherently non-negative — hence $\sum |\text{sim}|$.

(b) Suppose $\mu_u = \mu_v = 3$, the single neighbor rated $r_{vi} = 1$ (below their mean, so $r_{vi} - \mu_v = -2$), and $\text{sim}(u, v) = -0.5$:

$$\text{abs: } \hat{r} = 3 + \frac{(-0.5)(-2)}{0.5} = 3 + \frac{1}{0.5} = 3 + 2 = \mathbf{5},$$

$$\text{unsigned: } \hat{r} = 3 + \frac{(-0.5)(-2)}{-0.5} = 3 + \frac{1}{-0.5} = 3 - 2 = \mathbf{1}.$$

The neighbor has *opposite* taste ($\text{sim} < 0$) and *dislikes* the item ($r_{vi} < \mu_v$); opposite taste means “the neighbor’s dislike predicts the user’s like,” so the prediction should move *up* from the mean. The abs version does exactly that (predicts 5, above the user’s mean); the unsigned version cancels the two negatives and instead predicts 1 (below the mean), i.e. it treats an anti-correlated neighbor as if it were a positive one. The abs denominator is the correct, more sensible choice.

Rubric (3 pts) (a) instability/near-zero or negative denominator; (3 pts) (b) both computations; (2 pts) which is sensible and why.

Chapter 25

Part III — Latent-factor Collaborative Filtering: Solutions

The 5×5 matrix used in the ALS/CGD problems (“–” = empty) is

$$M = \begin{pmatrix} 5 & 2 & 4 & 4 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ - & - & 3 & 1 & 4 \\ 2 & 5 & 4 & 3 & 5 \\ 4 & 4 & 5 & 4 & - \end{pmatrix},$$

and the small matrix used in the objective/gradient/SGD problems is $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$ (all observed), $d = 1$.

Ex. 1 — UV decomposition and latent factors (A, Easy)

Solution. The UV decomposition writes $M \approx F_{\text{user}} F_{\text{item}}^T$, where M is $n \times k$ (users $\times$ items), F_{user} is $n \times d$, F_{item} is $k \times d$, and $d \ll \min(n, k)$ is the *latent dimension*. Multiplying the two factors reconstructs *every* entry $u_i \cdot v_j$, including the empty ones, because the same dot product that fits the observed cells also assigns a value to the unobserved cells — this is how the model “completes” the matrix.

Rubric (2 pts) equation; (2 pts) shapes; (1 pts) d ; (1 pts) completion mechanism.

Ex. 2 — Storage advantage (A, Easy)

Solution. (a) The completed matrix has $nk = 10^6 \cdot 10^6 = 10^{12}$ entries. (b) The two factor matrices hold $(n + k)d = (10^6 + 10^6) \cdot 50 = 2 \cdot 10^6 \cdot 50 = 10^8$ entries. The general comparison is $(n + k)d$ vs. nk ; here the factors need $\frac{10^{12}}{10^8} = 10^4$ times fewer numbers (three to four orders of magnitude). We store the factors and reconstruct any row on demand as $u_i V^T$.

Rubric (1 pts) (a); (2 pts) (b); (1 pts) general form; (1 pts) implication.

Ex. 3 — Batch GD vs. SGD (A, Moderate)

Solution. *Batch* GD computes the gradient using *all* observed ratings and takes one update per full pass ($U \leftarrow U + \alpha EV$); each update is expensive but the direction is an exact, low-variance descent direction. *SGD* updates after each single observed pair (i, j) using $u_i \leftarrow u_i + \alpha e_{ij} v_j$ and $v_j \leftarrow v_j + \alpha e_{ij} u_i$; each update is cheap and frequent, so it converges much faster, but the updates are noisy and can wander (and can escape poor local minima).

Rubric (2 pts) batch description; (2 pts) SGD description; (1 pts) cost; (1 pts) stability/speed tradeoff.

Ex. 4 — ALS vs. SGD vs. CGD (A, Moderate)

Solution. SGD is the most efficient (one cheap update per rating) but the least stable (needs a learning rate, noisy, can diverge). ALS is the most stable (each step is an exact least-squares solve, no learning rate, no divergence) but the most expensive (many linear systems per sweep). CGD sits between: it solves one *coordinate* in closed form at a time, so it is more stable than SGD and cheaper than full ALS. The sharpest difference between ALS and SGD is the *learning rate*: ALS needs none, SGD needs one to be chosen and tuned.

Rubric (2 pts) each algorithm's position; (2 pts) the learning-rate contrast.

Ex. 5 — Interpretation of latent factors (A, Moderate)

Solution. A latent factor is one coordinate of the hidden d -dimensional space in which users and items are jointly embedded. The factors are *latent* (hidden) because they are *not* observed attributes — they are discovered automatically by minimizing the objective, and the algorithm never labels them. They often end up correlated with interpretable qualities (“prefers action,” “dry humor”), but the model cannot attach human names automatically because it has no ground-truth meaning for the dimensions; any names are *post hoc* interpretations by a human.

Rubric (2 pts) definition; (2 pts) why latent; (1 pts) why no automatic names.

Ex. 6 — Objective value (B, Easy)

Solution. With $U = [1, 1]^\top$, $V = [1, 2]^\top$, the predictions $u_i v_j$ are:

	$j=1$	$j=2$
$i=1$	1	2
$i=2$	1	2

Residuals $r_{ij} - u_i v_j$: $(5 - 1) = 4$, $(2 - 2) = 0$, $(3 - 1) = 2$, $(4 - 2) = 2$; squared: 16, 0, 4, 4. Hence

$$J = \frac{1}{2}(16 + 0 + 4 + 4) = \frac{1}{2} \cdot 24 = \mathbf{12}.$$

Rubric (2 pts) predictions; (1 pts) residuals; (1 pts) value.

Common mistake. Forgetting the $\frac{1}{2}$, giving $J = 24$.

Ex. 7 — Gradient of one element (B, Moderate)

Solution. For $d = 1$, $\partial J / \partial u_1 = -\sum_j e_{1j} v_j$ (over user 0's rated items, both columns) and $\partial J / \partial v_1 = -\sum_i e_{i1} u_i$ (over item 0's rated users, both rows). The residuals are $e_{11} = 4$, $e_{12} = 0$, $e_{21} = 2$, $e_{22} = 2$. Thus

$$\begin{aligned}\frac{\partial J}{\partial u_1} &= -(e_{11}v_1 + e_{12}v_2) = -(4 \cdot 1 + 0 \cdot 2) = -4, \\ \frac{\partial J}{\partial v_1} &= -(e_{11}u_1 + e_{21}u_2) = -(4 \cdot 1 + 2 \cdot 1) = -6.\end{aligned}$$

Rubric (2 pts) $\partial J / \partial u_1$; (2 pts) $\partial J / \partial v_1$.

Common mistake. Using the wrong sign, or summing over the wrong index (items for v_1 instead of users).

Ex. 8 — One CGD coordinate ($d = 1$) (B, Moderate)

Solution. For $d = 1$ the CGD closed form collapses to the ratio $u = \sum_s v_s r_s / \sum_s v_s^2$ (with the $\sum_{t \neq j}$ term vanishing).

Fix $V = [1, 2]$ and optimize u_1 (user 0's ratings $r_{11} = 5$, $r_{12} = 2$):

$$u_1 = \frac{v_1 r_{11} + v_2 r_{12}}{v_1^2 + v_2^2} = \frac{1 \cdot 5 + 2 \cdot 2}{1 + 4} = \frac{9}{5} = \mathbf{1.8}.$$

Now fix $u_1 = 1.8$, $u_2 = 1$ and optimize v_1 (item 0's ratings $r_{11} = 5$, $r_{21} = 3$):

$$v_1 = \frac{u_1 r_{11} + u_2 r_{21}}{u_1^2 + u_2^2} = \frac{1.8 \cdot 5 + 1 \cdot 3}{1.8^2 + 1^2} = \frac{9 + 3}{3.24 + 1} = \frac{12}{4.24} \approx \mathbf{2.8302}.$$

Rubric (2 pts) $u_1 = 1.8$; (2 pts) $v_1 \approx 2.83$.

Ex. 9 — ALS first iteration (C, Moderate)

Solution. Step 1 (fix all-ones U , solve V): each item's two factors split its observed mean evenly. Item means: $0: \frac{5+3+2+4}{4} = 3.5$, $1: \frac{2+1+5+4}{4} = 3.0$, $2: \frac{4+2+3+4+5}{5} = 3.6$, $3: \frac{4+4+1+3+4}{5} = 3.2$, $4: \frac{3+1+4+5}{4} = 3.25$. Hence $v_0 = [1.75, 1.75]$, $v_1 = [1.5, 1.5]$, $v_2 = [1.8, 1.8]$, $v_3 = [1.6, 1.6]$, $v_4 = [1.625, 1.625]$.

Step 2 (fix V , solve U) for *user 3*, whose ratings are $[2, 5, 4, 3, 5]$. Let $s = u_{31} + u_{32}$; then

$$\begin{aligned}s &= \frac{\sum_j r_{3j} \frac{\bar{m}_j}{2}}{\sum_j (\frac{\bar{m}_j}{2})^2} = \frac{2(1.75) + 5(1.5) + 4(1.8) + 3(1.6) + 5(1.625)}{1.75^2 + 1.5^2 + 1.8^2 + 1.6^2 + 1.625^2} \\ &= \frac{3.5 + 7.5 + 7.2 + 4.8 + 8.125}{3.0625 + 2.25 + 3.24 + 2.56 + 2.640625} = \frac{31.125}{13.753125} = 2.263122,\end{aligned}$$

so $u_3 = [\frac{s}{2}, \frac{s}{2}] = [1.131561, 1.131561]$, matching the notebook's 1.13156101.

Rubric (2 pts) item means; (2 pts) five v_j ; (3 pts) user-3 factor (numerator, denominator, split).

Common mistake. Dividing by $d = 2$ inconsistently, or solving a full matrix instead of recognizing the all-ones collapse to a mean.

Ex. 10 — CGD first two coordinates (C, Moderate)

Solution. Optimize $u_{00} = x$ first. With $v_j = [1, 1]$ for all j , the predicted first row is $x + 1$ in every column, so the SSE is $(4-x)^2 + (1-x)^2 + (3-x)^2 + (3-x)^2 + (2-x)^2$. Differentiating: $-2[(4-x) + (1-x) + (3-x) + (3-x) + (2-x)] = 0 \Rightarrow 13 - 5x = 0 \Rightarrow x = \frac{13}{5} = \mathbf{2.6}$.

Then optimize $v_{00} = y$. Only column 0 depends on y ; users who rated item 0 are 0, 1, 3, 4 with ratings 5, 3, 2, 4 and user vectors $u_0 = [2.6, 1]$, $u_1 = u_3 = u_4 = [1, 1]$, so the predicted first column is $2.6y + 1$, $y + 1$, $y + 1$, $y + 1$ and the SSE is $(4 - 2.6y)^2 + (2 - y)^2 + (1 - y)^2 + (3 - y)^2$. Differentiating: $-2[2.6(4 - 2.6y) + (2 - y) + (1 - y) + (3 - y)] = 0 \Rightarrow 16.4 - 9.76y = 0 \Rightarrow y = \frac{16.4}{9.76} \approx \mathbf{1.68}$.

Rubric (2 pts) x SSE and derivative; (2 pts) $x = 2.6$; (2 pts) y SSE and derivative; (2 pts) $y \approx 1.68$.

Common mistake. Updating the whole row of U instead of only the single element u_{00} .

Ex. 11 — SGD one epoch (C, Moderate)

Solution. Start $u_1 = u_2 = 1$, $v_1 = 1$, $v_2 = 2$, $\alpha = 0.1$. Process $(1, 1)$, $(1, 2)$, $(2, 1)$, $(2, 2)$ (simultaneous updates: each uses pre-update values).

- $(1, 1)$: $e = 5 - 1 \cdot 1 = 4$; $u_1 \leftarrow 1 + 0.1 \cdot 4 \cdot 1 = 1.4$; $v_1 \leftarrow 1 + 0.1 \cdot 4 \cdot 1 = 1.4$.
- $(1, 2)$: $e = 2 - 1.4 \cdot 2 = -0.8$; $u_1 \leftarrow 1.4 + 0.1(-0.8)(2) = 1.24$; $v_2 \leftarrow 2 + 0.1(-0.8)(1.4) = 1.888$.
- $(2, 1)$: $e = 3 - 1 \cdot 1.4 = 1.6$; $u_2 \leftarrow 1 + 0.1(1.6)(1.4) = 1.224$; $v_1 \leftarrow 1.4 + 0.1(1.6)(1) = 1.56$.
- $(2, 2)$: $e = 4 - 1.224 \cdot 1.888 = 4 - 2.310912 = 1.689088$; $u_2 \leftarrow 1.224 + 0.1(1.689088)(1.888) = 1.224 + 0.318900 = 1.542900$; $v_2 \leftarrow 1.888 + 0.1(1.689088)(1.224) = 1.888 + 0.206744 = 2.094744$.

Final $U = [1.24, 1.5429]^\top$, $V = [1.56, 2.0947]^\top$.

Rubric (2 pts) first two updates; (2 pts) third; (2 pts) fourth; (1 pts) final state.

Common mistake. Reusing a just-updated value within the same entry's pair of updates instead of the pre-update values.

Ex. 12 — Factorize and recommend (D, Difficult)

Solution. For $d = 1$ the CGD closed form becomes the ALS equations: with V fixed, $u_i = \sum_j v_j r_{ij} / \sum_j v_j^2$; with U fixed, $v_j = \sum_i u_i r_{ij} / \sum_i u_i^2$. Sweeping these to convergence is a power-iteration on $MM^\top / M^\top M$ and converges (up to scale) to the *dominant* singular pair of M — i.e. the best rank-1 approximation of M in Frobenius norm.

For $M = \begin{pmatrix} 5 & 2 \\ 3 & 4 \end{pmatrix}$, the dominant singular value is $\sigma_1^2 = \frac{54 + \sqrt{2132}}{2} \approx 50.0868$ ($\sigma_1 \approx 7.0772$), with right singular vector $v \approx [0.8072, 0.5902]$ and $U = Mv \approx [5.2165, 4.7826]$. Thus (scale of u vs. v arbitrary):

$$U = \begin{pmatrix} 5.2165 \\ 4.7826 \end{pmatrix}, \quad V = \begin{pmatrix} 0.8072 \\ 0.5902 \end{pmatrix}, \quad \hat{M} = UV^\top \approx \begin{pmatrix} 4.2107 & 3.0788 \\ 3.8603 & 2.8227 \end{pmatrix}.$$

$\hat{M}$ has rank at most 1 because it is the *outer product of two vectors*: every row is a scalar multiple of $V^\top$, and every column a scalar multiple of U . The residual $\|M - \hat{M}\|_F^2 = \sigma_2^2 \approx 3.913$ is the irreducible error from discarding the second (small) component.

Rubric (3 pts) d=1 equations and power-iteration view; (2 pts) converged U, V ; (2 pts) $\hat{M}$; (2 pts) rank ≤ 1 explanation.

Ex. 13 — Derive the u_{ij} closed form (E, Difficult)

Solution. The SSE over user i 's rated items is

$$S = \sum_{s:r_{is} \neq \emptyset} \left(r_{is} - u_{ij}v_{sj} - \sum_{t \neq j} u_{it}v_{st} \right)^2.$$

Only the term $u_{ij}v_{sj}$ depends on u_{ij} , so

$$\frac{\partial S}{\partial u_{ij}} = \sum_{s:r_{is} \neq \emptyset} 2 \left(r_{is} - u_{ij}v_{sj} - \sum_{t \neq j} u_{it}v_{st} \right) (-v_{sj}).$$

Setting to zero and dividing by -2 :

$$\sum_s v_{sj} \left(r_{is} - \sum_{t \neq j} u_{it}v_{st} \right) - u_{ij} \sum_s v_{sj}^2 = 0,$$

$$u_{ij} = \frac{\sum_{s:r_{is} \neq \emptyset} v_{sj} \left(r_{is} - \sum_{t \neq j} u_{it}v_{st} \right)}{\sum_{s:r_{is} \neq \emptyset} v_{sj}^2}.$$

Rubric (3 pts) derivative; (3 pts) rearrange to closed form.

Ex. 14 — All-ones ALS and the minimum norm (E, Difficult)

Solution. When U is all ones, the predicted rating for item j is $u_i \cdot v_j = v_{j1} + v_{j2} + \dots + v_{jd}$, which depends only on the *sum* of item j 's d factors, not on how that sum is split across factors. The least-squares problem for v_j is therefore under-determined: infinitely many vectors v_j share the same sum s_j and fit the data identically. The normal equations reflect this — with all rows of U identical, the matrix $\sum_i u_i u_i^\top$ is rank-1 (rank-deficient for $d > 1$), so the system has no unique solution.

The notebook's solver (`scipy.linalg.lstsq`) returns the *minimum-norm* solution among these, which is the one that spreads the sum *evenly*: $v_{jq} = s_j/d$ for every q . Since the optimal sum s_j equals the item's observed mean $\bar{m}_j$ (the best constant prediction), each factor becomes $\bar{m}_j/d$ — precisely the “item mean divided by d ” rule stated in the problem.

Rubric (3 pts) why only the sum matters; (2 pts) rank deficiency; (2 pts) minimum-norm $\Rightarrow$ even split $\bar{m}_j/d$.

Formula Sheet (Master)

Part I — Frequent Itemsets and Association Rules

Absolute and relative support

$\text{sup}(X) = |\{T_i \in T : X \subseteq T_i\}|$, $\text{relSup}(X) = \frac{\text{sup}(X)}{N}$. An itemset X is *frequent* iff $\text{sup}(X) \geq \text{minsup}$.

Support monotonicity (downward closure)

If $Y \subseteq X$ then $\text{sup}(Y) \geq \text{sup}(X)$.

Confidence and lift of rule $A \rightarrow B$

$\text{conf}(A \rightarrow B) = \frac{\text{sup}(A \cup B)}{\text{sup}(A)}$, $\text{lift}(A \rightarrow B) = \frac{\text{conf}(A \rightarrow B)}{\text{relSup}(B)}$.

Brute-force complexity

$O(N \cdot 2^d)$ support computations over all $2^d - 1$ non-empty itemsets.

Part II — Neighborhood-based Collaborative Filtering

Raw cosine over common items

$\text{RawCosine}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2} \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}$.

Adjusted cosine = Pearson correlation

$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)(r_{vi} - \mu_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \mu_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \mu_v)^2}}$.

Weighted-average prediction

$\hat{r}_{ui} = \mu_u + \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v)(r_{vi} - \mu_v)}{\sum_{v \in N_i^k(u)} |\text{sim}(u, v)|}$.

DCG and NDCG

$$\text{DCG} = \frac{1}{m} \sum_{u=1}^m \sum_{\substack{j \in I_u \\ v_j \leq L}} \frac{2^{r_{uj}} - 1}{\log_2(v_j + 1)}, \quad \text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}.$$

Part III — Latent-factor Collaborative Filtering**UV decomposition and objective**

$$M \approx F_{\text{user}} F_{\text{item}}^\top, \quad J = \frac{1}{2} \|M - F_{\text{user}} F_{\text{item}}^\top\|_F^2 = \frac{1}{2} \sum_{(i,j): r_{ij} \neq \emptyset} (r_{ij} - \sum_s u_{is} v_{js})^2.$$

Gradient-descent update (batch)

$$e_{ij} = r_{ij} - \sum_s u_{is} v_{js}; \quad U \leftarrow U + \alpha E V; \quad V \leftarrow V + \alpha E^\top U.$$

ALS per-row solve

For each user i : $u_i \leftarrow \arg \min_u \frac{1}{2} \sum_{j: r_{ij} \neq \emptyset} (r_{ij} - u v_j)^2$ (least squares); symmetric for each item j .

CGD per-element closed form

$$u_{ij} = \frac{\sum_{s: r_{is} \neq \emptyset} v_{sj} (r_{is} - \sum_{t \neq j} u_{it} v_{st})}{\sum_{s: r_{is} \neq \emptyset} v_{sj}^2}; \quad v_{ij} = \frac{\sum_{s: r_{si} \neq \emptyset} u_{sj} (r_{si} - \sum_{t \neq j} u_{st} v_{it})}{\sum_{s: r_{si} \neq \emptyset} u_{sj}^2}.$$

Bibliography

- [1] C. C. Aggarwal, *Data Mining: The Textbook*, Springer, 2015.
- [2] P. Fournier-Viger, J. C.-W. Lin, B. Vo, T. T. Chi, J. Zhang, H. B. Le, *A Survey of Itemset Mining*, Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 2017.
- [3] U. Malik, *Association Rule Mining via Apriori Algorithm in Python*, StackAbuse, 2018.
- [4] J. Leskovec, A. Rajaraman, J. Ullman, *Mining of Massive Datasets*, 3rd edition, Cambridge University Press.
- [5] C. C. Aggarwal, *Recommender Systems: The Textbook*, Springer, 2016.
- [6] C. Alis, course notebooks: *Frequent Itemset Mining, Neighborhood-based Collaborative Filtering Exercises, Latent factor-based Collaborative Filtering Exercises*.
- [7] Course lecture slides, Sessions 3–10 (Frequent Itemset Mining & Association Rules; Neighborhood-based CF; Latent-factor CF).